Desarrollo de un prototipo de aplicación móvil de análisis automatizado de URLs extraídos de códigos QR mediante análisis aislado: Módulo de escaneo y diseño de interfaz de usuario con filtro de consulta estática a APIs de detección de amenazas

María José Villafuerte Arredondo



UVG UNIVERSIDAD DEL VALLE DE GUATEMALA

UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



Desarrollo de un prototipo de aplicación móvil de análisis automatizado de URLs extraídos de códigos QR mediante análisis aislado: Módulo de escaneo y diseño de interfaz de usuario con filtro de consulta estática a APIs de detección de amenazas

Trabajo de graduación en modalidad de trabajo profesional presentado por

María José Villafuerte Arredondo

Para optar al grado académico de Licenciado en Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Guatemala, Noviembre del 2026

Vo.Bo.:

(f) _____

Ing. Juan Carlos Canteo

Tribunal Examinador:

(f) _____

Ing. Juan Carlos Canteo

(f) _____

MSc. Carlos Juárez

(f) _____

Ing. Luis Pedro Gonzáles

Fecha de aprobación: Guatemala, _____ de _____ de 2026.

Prefacio

Siempre me han parecido muy interesantes los avances tecnológicos, y también los riesgos a los que nos exponemos con ellos sin siquiera saberlo. Cuando descubrí que a través de un simple enlace es posible inyectar código o extraer datos de un dispositivo, me pareció algo sorprendente. Recuerdo haber pensado en lo ingenioso que tenía que ser alguien para crear algo así, y en lo extraño que era que todo ese ingenio se usara para hacer daño.

Desde entonces adopté una costumbre: cada vez que no estoy segura de un enlace, lo paso por VirusTotal antes de abrirlo. Un día, en una conferencia de ciberseguridad, escuché que los teléfonos no tienen una protección directa sobre los códigos QR, que simplemente los leen y abren lo que traen dentro. Me puse como misión encontrar una herramienta que hiciera con los QR lo que VirusTotal hace con los enlaces, y no encontré nada tan sólido.

La idea se quedó conmigo. Cada vez que veía un código QR en un restaurante, en un parqueo o en un cartel, pensaba en lo complicado que sería tomarle una foto, pasarla por un lector, extraer el enlace y recién entonces revisarlo. Y además, ¿qué pasaría si el enlace era malicioso pero todavía nadie lo había reportado? Ninguna lista lo iba a detectar.

Mi inspiración para la forma de la aplicación fue el lector de códigos QR que trae el iPhone en su centro de control, junto a los botones de sonido y de brillo. Lo uso mucho, y justamente por eso soy consciente de que no revisa nada. Me imaginé una herramienta igual de simple y de rápida de abrir, pero con la misión de leer los códigos de forma segura y de decirme a dónde me llevan antes de que yo llegue ahí.

Así nació QRisk: de una curiosidad, de un temor y de una necesidad personal de tener un escudo entre las nuevas tecnologías y nuestra seguridad. Este trabajo documenta el camino de convertir esa idea en un prototipo funcional.

Agradecimientos

Quiero agradecer, en primer lugar, a [nombres de mi familia], por su apoyo durante toda la carrera y por la paciencia que tuvieron conmigo en los meses de este proyecto.

A mi asesor, el Ing. Juan Carlos Canteo, por su guía, por sus observaciones durante las revisiones del protocolo y por ayudarme a mantener el alcance del trabajo en algo que se pudiera terminar y defender.

A Fabiola Contreras, mi compañera en este proyecto, quien desarrolló el motor de análisis del lado del servidor. Sin su trabajo en la cascada de filtros, la aplicación no tendría un veredicto que mostrar. Coordinar dos módulos en paralelo no siempre fue sencillo, y agradezco su disposición para acordar cada detalle del contrato entre ambos.

A la Universidad del Valle de Guatemala y a los catedráticos del Departamento de Ciencias de la Computación, por la formación que hizo posible este trabajo.

A [nombres de amigos o personas que participaron en las pruebas], por prestarme su tiempo y su paciencia para probar la aplicación.

Índice

Prefacio III

Agradecimientos IV

Lista de Figuras IX

Lista de Cuadros X

Resumen XI

1. Introducción 1
- 1.1. Definición del problema . . . 1
- 1.2. La propuesta: QRisk . . . 2
- 1.3. Organización del documento . . . 2

2. Objetivos 3
- 2.1. Objetivo general . . . 3
- 2.2. Objetivos específicos . . . 3

3. Justificación 4
- 3.1. Por qué intervenir en el escaneo . . . 4
- 3.2. A quién le sirve este trabajo . . . 5

4. Marco Teórico 6
- 4.1. Amenazas de seguridad en el entorno móvil . . . 6
  - 4.1.1. Phishing . . . 6
  - 4.1.2. Quishing: phishing mediante códigos QR . . . 6
  - 4.1.3. Ingeniería social y susceptibilidad del usuario móvil . . . 7
  - 4.1.4. Análisis de URLs maliciosas . . . 7
- 4.2. El estándar QR y su explotación como vector de ataque . . . 7
  - 4.2.1. Fundamentos técnicos del código QR . . . 7
  - 4.2.2. Tipos de contenido codificable . . . 8
  - 4.2.3. El QR como vector de ataque . . . 8
- 4.3. Sistemas de detección y clasificación de amenazas . . . 8
  - 4.3.1. Listas negras y sistemas de reputación . . . 8
  - 4.3.2. Threat intelligence colaborativa . . . 8
  - 4.3.3. Análisis heurístico de URLs . . . 9
  - 4.3.4. Aprendizaje automático aplicado a URLs . . . 9

4.3.5. Métricas de evaluación de clasificadores . . . 9
4.4. Escaneo y procesamiento nativo en dispositivos móviles . . . 9
4.4.1. APIs nativas de detección de códigos . . . 9
4.4.2. Librerías de cámara para React Native . . . 10
4.4.3. Latencia de decodificación . . . 10
4.5. Desarrollo móvil multiplataforma con React Native . . . 10
4.5.1. Arquitectura de React Native . . . 10
4.5.2. Expo y el desarrollo desde una sola base de código . . . 10
4.6. Seguridad en aplicaciones móviles . . . 11
4.6.1. OWASP MASVS . . . 11
4.6.2. Seguridad de red en iOS y Android . . . 11
4.6.3. Almacenamiento local . . . 11
4.7. Fuentes de reputación del filtro F1 . . . 11
4.7.1. Google Safe Browsing . . . 11
4.7.2. PhishTank . . . 11
4.7.3. VirusTotal . . . 12
4.7.4. URLhaus y MISP . . . 12
4.8. Geolocalización y geocodificación inversa . . . 12
4.9. Usabilidad y seguridad centrada en el usuario . . . 12
4.9.1. Usable security . . . 12
4.9.2. Efectividad de las advertencias . . . 13
4.10. Arquitectura de backend en la nube . . . 13

5. Antecedentes **14**
5.1. Procedimiento de la revisión . . . 14
5.2. Revisión de antecedentes . . . 15
5.2.1. El código QR como vector de ataque . . . 15
5.2.2. Detección de URLs maliciosas . . . 15
5.2.3. Sistemas de reputación y threat intelligence . . . 15
5.2.4. Soluciones comerciales de escaneo seguro . . . 16
5.2.5. Usable security y efectividad de advertencias . . . 16
5.3. Comparación de los trabajos revisados . . . 16
5.4. Limitaciones identificadas en la literatura . . . 17
5.5. Fundamentación del proyecto . . . 17

6. Alcance **18**
6.1. Definición del alcance . . . 18
6.2. Delimitación funcional y técnica . . . 19
6.3. Restricciones . . . 19
6.4. Exclusiones explícitas . . . 19

7. Metodología **21**
7.1. Enfoque del trabajo . . . 21
7.2. Metodología de gestión . . . 21
7.3. Fases del proyecto . . . 22
7.3.1. Reorganización por bloques de trabajo . . . 22
7.3.2. El contrato de interfaz como instrumento . . . 23
7.4. Estrategia de validación . . . 23
7.4.1. Nivel 1: pruebas automatizadas de la lógica de negocio . . . 23
7.4.2. Nivel 2: verificación en dispositivo y contra servicios reales . . . 24
7.4.3. Nivel 3: pruebas de usabilidad . . . 24
7.5. Diseño de las mediciones . . . 24
7.5.1. Tiempo de lectura y tasa de detección . . . 24
7.5.2. Mediciones de los filtros de análisis . . . 25

7.5.3. Protocolo de verificación en dispositivo . . . 26
7.5.4. Métricas e instrumentos . . . 26
7.6. Consideraciones éticas y de privacidad . . . 27
7.7. Limitaciones del método . . . 27

8. Desarrollo del prototipo **28**
8.1. Visión general del flujo . . . 28
8.2. Plataforma y versiones . . . 29
8.3. Arquitectura de la aplicación . . . 30
8.4. Escaneo y clasificación del contenido . . . 30
8.5. El filtro F1 en el dispositivo . . . 30
8.6. Integración con la cascada del servidor . . . 31
8.7. Determinación del veredicto . . . 32
8.8. Política de tiempos y degradación . . . 32
8.9. Pantallas de resultado . . . 33
8.10. Abrir el sitio analizado . . . 34
8.11. Historial local . . . 34
8.12. Reporte de códigos maliciosos . . . 35
8.13. Story Card . . . 35
8.14. Geocodificación inversa . . . 35
8.15. Diferencias entre el protocolo y lo implementado . . . 35

9. Resultados **37**
9.1. Verificación de la lógica de negocio . . . 37
9.2. Compilación reproducible . . . 38
9.3. Filtro F1 de reputación . . . 38
9.3.1. Verificación funcional en vivo . . . 38
9.3.2. F1 sobre la muestra de Kaggle . . . 39
9.3.3. Validez de la muestra de Kaggle . . . 39
9.4. Cascada completa de filtros . . . 40
9.4.1. Tiempo total y cortes del servidor . . . 40
9.4.2. Cobertura de cada filtro sobre amenazas activas . . . 40
9.4.3. Elevación por evidencia de comportamiento . . . 41
9.4.4. Detección sobre la muestra de Kaggle . . . 41
9.5. Detección con una muestra verificada . . . 42
9.6. Lectura de códigos QR . . . 42
9.7. Verificación en el dispositivo . . . 44
9.8. Usabilidad . . . 44
9.9. Cumplimiento de las métricas . . . 45
9.10. Limitaciones de los resultados . . . 45

10. Conclusiones **47**

11. Recomendaciones **49**

Bibliografía **54**

Anexos **55**

A. Evidencia de las mediciones **55**

B. Resumen del contrato de interfaz **56**
B.1. Análisis de una URL . . . 56
B.2. Reporte de un código . . . 57

C. Códigos QR de prueba **58**

D. Protocolo de verificación en dispositivo **59**

Glosario **60**

Lista de Símbolos **62**

Lista de Figuras

8.1. Recorrido de un código QR dentro de QRisk 29

9.1. Distribución de T_{total} en las respuestas exitosas y proporción de cortes por HTTP 504 40
9.2. Histograma de T_{scan} con la línea del criterio de 5 s 43
9.3. Distribución de T_{scan} por condición de tamaño, ángulo e iluminación 43
9.4. Pantallas de resultado para los veredictos seguro, sospechoso y malicioso 44

Lista de Cuadros

5.1. Comparación de enfoques según su punto de intervención 17

6.1. Componentes incluidos en el módulo 19

7.1. Fases planificadas en el protocolo 22
7.2. Bloques de desarrollo y criterio de cierre 23
7.3. Composición de referencia de la muestra de lectura 25
7.4. Métricas del módulo y su instrumento de medición 26

8.1. Versiones del entorno de desarrollo 29
8.2. Capas de la aplicación 30
8.3. Bandas de puntaje y su fundamento 32
8.4. Evolución de la latencia del servidor y del tiempo límite 33
8.5. Veredictos y su significado para el usuario 34
8.6. Diferencias con el protocolo y su justificación 36

9.1. Métricas verificadas con pruebas automatizadas (11 de octubre de 2026) 37
9.2. Tiempo del filtro F1 sobre muestras de Kaggle (12 de octubre de 2026) 39
9.3. Qué filtro detecta cada tipo de amenaza (12 de octubre de 2026) 41
9.4. Métricas de detección sobre la muestra de Kaggle, sin corregir etiquetas 41
9.5. Detección sobre la muestra verificada (por completar) 42
9.6. Resultados globales de lectura (por completar) 42
9.7. Resultados de lectura por condición (por completar) 43
9.8. Resultados del protocolo en dispositivo (por completar) 44
9.9. Registro de la prueba de usabilidad (por completar) 44
9.10. Cumplimiento de las métricas del protocolo 45

A.1. Ubicación de la evidencia en el repositorio 55

C.1. Contenidos de ejemplo y pantalla esperada 58
C.2. Códigos de demostración de los veredictos 58

Resumen

El código QR se volvió parte de la vida cotidiana, pero su contenido no puede leerse a simple vista y los teléfonos lo abren sin revisarlo. Esa combinación es la que aprovecha el *quishing*, una variante del phishing en la que un código manipulado lleva al usuario hacia un sitio fraudulento sin que este pueda ver el destino antes de llegar. El presente trabajo describe el diseño, desarrollo y validación del módulo de escaneo e interfaz de usuario de QRisk, un prototipo de aplicación móvil que se coloca entre el escaneo y la apertura del enlace para analizar la URL antes de que el usuario la visite.

El módulo se construyó con React Native sobre Expo SDK 57 y se entregó para Android. Lee el código con la cámara del dispositivo, clasifica su contenido para separar los enlaces del resto, resuelve en el propio teléfono el primer filtro de reputación (una lista local de dominios confiables y consultas en paralelo a Google Safe Browsing y PhishTank) y envía el resultado al motor de análisis del servidor, desarrollado en paralelo, que completa una cascada de filtros heurísticos y de comportamiento. El veredicto se presenta en cuatro categorías (seguro, sospechoso, malicioso e incierto) con dos niveles de detalle. El módulo incluye además un historial local que funciona sin conexión, el reporte anónimo de códigos maliciosos y una tarjeta compartible que advierte a otras personas sobre un código peligroso.

La lógica de negocio quedó cubierta por pruebas automatizadas muy por encima del criterio del 70 % establecido en el protocolo. Las pruebas sobre una muestra pública de URLs mostraron que el primer filtro responde en fracciones de segundo y que cada filtro de la cascada detecta amenazas que los otros no ven. Las mediciones en dispositivo físico y las pruebas con usuarios completan la evaluación del prototipo.

Palabras clave: quishing, códigos QR, phishing, seguridad móvil, React Native, threat intelligence, usable security.

Pendiente: Ajustar el último párrafo del resumen cuando estén las mediciones del banco de pruebas automatizado, la verificación en dispositivo y la prueba de usabilidad.

CAPÍTULO 1

Introducción

Una persona común, al ver un código QR, no piensa nada malo sobre él. En primer lugar, porque no tiene forma de leerlo ni de interpretarlo a simple vista; en segundo lugar, porque ya se acostumbró a encontrarlo en todas partes. Ocurre algo parecido a lo que pasa con un cuchillo en una cocina: de tanto usarlo para cocinar, se olvida que también puede ser un arma. El código QR se percibe como una herramienta de acceso rápido al menú de un restaurante, al pago de un parqueo o al formulario de una institución, y casi nunca como un posible vector de ataque.

Esa confianza tiene un costo. El código QR fue diseñado en 1994 para rastrear componentes en la industria automotriz y está estandarizado en la norma ISO/IEC 18004 [1, 2]. Su adopción masiva convirtió un mecanismo industrial en una puerta de entrada cotidiana a contenido digital, presente en comercios, transporte, plataformas de gobierno y sistemas de pago [3]. Sin embargo, el proceso de escaneo no muestra al usuario el destino del enlace antes de abrirlo. A diferencia de un hipervínculo en pantalla, que puede inspeccionarse antes de tocarlo, el contenido de un QR permanece oculto hasta que el teléfono ya lo está procesando, y la mayoría de los sistemas operativos móviles abren la URL sin aplicar ningún análisis de seguridad [3, 4].

1.1. Definición del problema

Los actores maliciosos aprovechan esta dinámica mediante el *quishing*, término que combina QR y phishing. La técnica consiste en distribuir códigos QR manipulados que llevan a sitios fraudulentos, a la descarga de software malicioso o a formularios que capturan credenciales [5, 6]. Frente al phishing tradicional tiene tres ventajas para el atacante. El enlace no es visible antes del escaneo, por lo que el usuario pierde las señales de alerta que le daría una URL sospechosa. Los filtros de correo electrónico, que analizan texto pero no el contenido de una imagen, no lo detectan [6]. Y puede desplegarse en el mundo físico con una simple calcomanía pegada encima de un código legítimo.

El problema dejó de ser marginal. Insikt Group documentó un crecimiento cercano al 248 % en las menciones de quishing en fuentes de inteligencia entre el tercer y el cuarto trimestre de 2023, y registró su integración en plataformas de *phishing-as-a-service* como Tycoon 2FA y Greatness [5]. Que el ataque se venda como servicio indica que se profesionalizó y que ya no requiere conocimiento técnico para ejecutarlo.

Las deficiencias del ecosistema actual pueden identificarse en dos niveles. A nivel del usuario,

existe una percepción generalizada de los códigos QR como elementos neutros o seguros, lo que reduce la vigilancia ante una posible manipulación [3]. A nivel tecnológico, los lectores integrados en los sistemas operativos no incorporan ningún análisis del contenido antes de redirigir, de modo que toda la carga de detectar el engaño recae en la persona que escanea.

Existen aplicaciones que intentan cubrir este vacío. Herramientas como *Safe QR Scan* y *SafeQR Reader*, disponibles en las tiendas de aplicaciones, verifican la URL antes de abrirla, pero su mecanismo central consiste en contrastar el enlace contra listas negras de dominios conocidos. Este enfoque tiene una limitación documentada desde hace más de una década: las listas negras cubren una fracción pequeña de los sitios de phishing en las primeras horas de vida del ataque, justo cuando ocurre la mayor parte del daño [7]. En el contexto del quishing, donde el atacante puede registrar un dominio nuevo para cada campaña, depender solo de lo que ya fue reportado deja pasar precisamente los ataques más recientes.

Entonces, el problema que aborda este trabajo puede plantearse así: un usuario de teléfono inteligente no tiene forma de saber a dónde lo lleva un código QR antes de que su dispositivo lo abra, y las soluciones disponibles dependen casi por completo de listas que no conocen las amenazas nuevas.

1.2. La propuesta: QRisk

QRisk es un prototipo de aplicación móvil que se coloca en medio de ese intervalo. Lee el código QR, muestra a dónde apunta en realidad, lo analiza mediante una cascada de filtros y devuelve un veredicto en lenguaje sencillo antes de que el usuario decida si abre el sitio. La regla que guía todo el diseño es que la aplicación nunca visita la URL por su cuenta: hasta que la persona lo pida, el enlace es solo texto en pantalla.

El sistema se desarrolló entre dos personas, cada una a cargo de un módulo. El módulo de servidor, a cargo de Fabiola Contreras, implementa el motor de análisis en la nube con los filtros heurístico (F2.1), de comportamiento (F2.2) y de vista previa visual (F2.3). El módulo que documenta este trabajo, a cargo de la autora, comprende la aplicación móvil completa: el escaneo y la clasificación del contenido, la integración del filtro de reputación F1, el flujo de presentación del veredicto, el historial local, el reporte de códigos maliciosos y la tarjeta compartible de advertencia, conocida en el proyecto como *Story Card*.

1.3. Organización del documento

Continuando con esta introducción, el capítulo 2 presenta los objetivos aprobados en el protocolo, y el capítulo de justificación explica por qué vale la pena intervenir en el momento del escaneo. El marco teórico reúne los conceptos de seguridad, de desarrollo móvil y de usabilidad que sustentan las decisiones del prototipo, mientras que los antecedentes revisan los trabajos previos y delimitan el vacío que el proyecto busca cubrir. El capítulo de alcance fija lo que el trabajo incluye y lo que deja fuera.

Los capítulos centrales describen la metodología seguida, el desarrollo del prototipo con sus decisiones técnicas y los resultados obtenidos. El documento cierra con las conclusiones, las recomendaciones para trabajo futuro y los anexos con la evidencia de las mediciones.

CAPÍTULO 2

Objetivos

Los objetivos se mantienen tal como fueron aprobados en el protocolo del trabajo de graduación. Las decisiones técnicas que durante el desarrollo modificaron la forma de alcanzarlos, como el cambio de librería de escaneo o de almacenamiento, se explican y justifican en el capítulo de desarrollo del prototipo.

2.1. Objetivo general

Reducir el riesgo de acceso a sitios maliciosos codificados en códigos QR por usuarios móviles, mediante el desarrollo de una interfaz multiplataforma en React Native que integra el flujo completo de escaneo, presentación de resultados y consulta a las APIs de threat intelligence del primer filtro.

2.2. Objetivos específicos

1. Implementar el módulo de escaneo de códigos QR con extracción de URL en menos de 5 segundos y clasificación del tipo de contenido, mediante `react-native-vision-camera` con CodeScanner nativo en iOS y Android.
2. Desarrollar el flujo de interacción principal entre el escaneo y la presentación del veredicto, mediante la implementación de la pantalla de carga con animación skeleton/shimmer y la pantalla de resultado con dos niveles de detalle (simple y extendido).
3. Implementar las funcionalidades secundarias de historial offline, reporte y alerta social, mediante almacenamiento local sin cuenta de usuario, integración con el endpoint de reportes y generación de Story Card con geocodificación inversa.
4. Integrar el filtro F1 de consulta a listas de reputación con tiempo de respuesta inferior a 5 segundos, mediante la conexión con las APIs de Google Safe Browsing, PhishTank y VirusTotal como primer filtro de análisis.

CAPÍTULO 3

Justificación

Los códigos QR se volvieron parte de la rutina mucho más rápido de lo que avanzaron las medidas de seguridad que deberían acompañarlos. Como se mencionó anteriormente, cuando alguien escanea un código no ve a dónde lo lleva el enlace antes de que el teléfono lo abra, y los lectores integrados en los sistemas operativos hacen esa redirección sin revisar nada. Ese desbalance es lo que vuelve tan efectivo al quishing: el riesgo existe, pero es invisible justo en el momento en que el usuario tiene que decidir.

Hay tres razones por las que atender este problema ahora. La primera es que el ataque se industrializó al integrarse a plataformas de phishing como servicio [5]. La segunda es que los filtros de correo corporativo no leen los códigos QR incrustados como imagen, de modo que este vector los esquiva por completo [6]. La tercera es que las aplicaciones de escaneo seguro existentes dependen casi todas de listas negras, un enfoque que deja fuera a los dominios recién creados [7, 8, 9].

3.1. Por qué intervenir en el escaneo

Podría pensarse que el usuario ya está protegido, porque los navegadores móviles incorporan servicios de reputación de URLs y muestran advertencias ante sitios peligrosos. Esa objeción es válida y de su respuesta depende que este trabajo tenga sentido.

El navegador avisa tarde. Para cuando muestra la advertencia, la petición al servidor del atacante ya salió: la dirección IP del dispositivo, el modelo de navegador y todo lo que venía codificado en la URL ya fueron enviados. En una campaña de quishing, parte de lo que el atacante busca lo obtiene en ese primer contacto, sin que el usuario tenga que hacer nada más. Si el análisis ocurre entre la lectura del código y la apertura del navegador, el control se adelanta un paso: la URL ya se puede leer, pero todavía no hubo contacto con el atacante. Por esa razón la solución vive en la aplicación de escaneo y no más adelante en la cadena.

De esa decisión se desprenden las demás. La lectura del código se resuelve en el propio teléfono, sin enviar la imagen a ningún servidor. El análisis empieza por las fuentes de reputación, que dan la mayor certeza sobre amenazas conocidas con muy poco tiempo de respuesta [10, 11]. Que esas fuentes no alcancen para los dominios nuevos es justamente la razón por la que existen filtros posteriores.

3.2. A quién le sirve este trabajo

Para los usuarios de teléfonos inteligentes, una herramienta que revise el enlace antes de abrirlo atiende un riesgo que aparece en situaciones de todos los días: comercios, transporte público, oficinas de gobierno y pagos. El quishing afecta a cualquier persona con un teléfono, tenga o no conocimientos técnicos [4], por lo que el valor de la herramienta está en que no le exige al usuario saber nada especial para tomar una buena decisión.

En este punto es importante precisar hasta dónde llega el aporte. Lo que este módulo resuelve es convertir un veredicto técnico en una decisión que la persona pueda tomar, en el momento y en el dispositivo donde le toca tomarla. Eso se evalúa con criterios propios: si el veredicto se entiende, si el flujo se completa sin ayuda y si el tiempo de respuesta es aceptable. Son criterios distintos de los que miden qué tan bien detecta el motor de análisis, y ambos son necesarios. Una pantalla muy clara que comunica un veredicto poco confiable no protege a nadie, y un veredicto correcto que nadie entiende tampoco.

Para los negocios e instituciones que usan códigos QR para atender al público, el trabajo documenta un riesgo que afecta directamente la confianza en ese canal. Que alguien pegue una calcomanía encima del código legítimo de un establecimiento [3] es un problema que el establecimiento no puede resolver por su cuenta, y contar con evidencia sobre cómo verificar un enlace antes de abrirlo da una base para decidir qué medidas tomar.

Desde una perspectiva teórica, el trabajo contribuye al estudio de los ataques que aprovechan interfaces poco convencionales, en las que la señal que normalmente usaría el usuario para desconfiar, es decir, ver la URL, desaparece por el diseño mismo del medio [4, 3]. Es además un terreno poco explorado en América Latina, donde la adopción tecnológica ha avanzado más rápido que las prácticas de seguridad asociadas.

Para la Universidad del Valle de Guatemala, el trabajo suma en un área con poca producción académica local, la ciberseguridad aplicada, y deja un antecedente para futuros proyectos de seguridad móvil y análisis de amenazas. Para la autora, el proyecto significó afianzar conocimientos en seguridad informática, desarrollo móvil e integración de servicios de *threat intelligence*, trabajando sobre un problema con implicaciones éticas reales: qué se le advierte al usuario, con cuánto detalle y con cuánta certeza son decisiones que afectan directamente su seguridad.

Por lo tanto, el trabajo se justifica porque coinciden una amenaza que crece y que ya se volvió un negocio, un conjunto de soluciones con limitaciones técnicas documentadas y una población expuesta que no tiene protección en el momento que más importa. Intervenir desde la aplicación móvil, el único punto donde la URL puede leerse antes de que exista contacto con el atacante, tiene sentido tanto por su ubicación en la cadena del ataque como porque permite evaluar la propuesta con criterios concretos.

CAPÍTULO 4

Marco Teórico

Este capítulo reúne los conceptos que sustentan el prototipo. Empieza por la amenaza que lo motiva, continúa con el estándar QR y con las técnicas para detectar URLs maliciosas, y termina con las tecnologías móviles, las fuentes de reputación y los principios de usabilidad que guiaron las decisiones de diseño. En cada sección se explica, además del concepto, por qué es relevante para el trabajo.

4.1. Amenazas de seguridad en el entorno móvil

4.1.1. Phishing

El phishing es una forma de ingeniería social en la que un atacante suplanta a una entidad de confianza para que la víctima revele credenciales, datos financieros u otra información sensible, o para dirigirla hacia contenido malicioso. El nombre viene de la analogía con la pesca: el atacante lanza un anzuelo, que suele ser un mensaje o un enlace falso, y espera a que la víctima muerda. Aunque existe desde los años noventa, su volumen y su sofisticación no han dejado de crecer, porque se adapta a cada canal de comunicación nuevo que adoptan las personas.

El Anti-Phishing Working Group registró más de un millón de ataques de phishing únicos en el primer trimestre de 2024, con una tendencia que se mantuvo en 2025 [8, 9]. El informe de Verizon de 2024 señala al phishing como uno de los vectores de acceso inicial más frecuentes en los incidentes de seguridad a nivel mundial [12]. Estos datos confirman que se trata de una amenaza activa y no de un riesgo teórico.

4.1.2. Quishing: phishing mediante códigos QR

El quishing es el phishing que usa un código QR para distribuir el enlace malicioso. En el phishing tradicional la URL fraudulenta está a la vista en el cuerpo del mensaje; en el quishing queda escondida dentro del patrón visual del código, así que el usuario no puede inspeccionarla directamente [5, 6].

Un ataque típico sigue cuatro pasos. Primero, el atacante genera un QR que codifica una URL bajo su control. Después lo coloca en un canal donde la gente confía de forma implícita: un correo

corporativo, un cartel, una factura o el menú de un restaurante. La víctima lo escanea con la cámara del teléfono y, por último, el navegador abre la URL sin que nadie la haya evaluado [4].

Vidas et al. mostraron en un estudio empírico que los usuarios de teléfonos inteligentes son más susceptibles al phishing por QR que al phishing por URL directa, precisamente porque el código elimina la señal visual que representa una dirección sospechosa [4]. Todo el flujo de QRisk responde a esta anatomía: la aplicación introduce un punto de inspección entre el tercer y el cuarto paso.

4.1.3. Ingeniería social y susceptibilidad del usuario móvil

La ingeniería social agrupa las técnicas psicológicas con las que un atacante manipula a una persona para que revele información o haga algo que comprometa la seguridad de un sistema. A diferencia de un ataque puramente técnico, explota sesgos humanos como la urgencia, la autoridad y la confianza [13]. En el quishing, el código aprovecha la confianza del lugar donde aparece: un QR en el mostrador de un banco proyecta la misma autoridad que el banco.

El contexto móvil agrava esta susceptibilidad. Escanear y abrir la URL es casi un reflejo, la pantalla pequeña dificulta revisar la barra de direcciones y el comportamiento habitual frente a un QR no incluye verificar el destino [4, 3]. Esto significa que la vulnerabilidad no es solo un descuido individual, sino una condición del medio que la solución técnica tiene que compensar. De aquí se derivan las decisiones sobre el momento, el tono y el formato en que se presenta el veredicto.

4.1.4. Análisis de URLs maliciosas

Sahoo, Liu y Hoi clasifican las técnicas de detección de URLs maliciosas en tres familias [14]. Las basadas en listas negras consultan bases de datos de URLs ya identificadas como maliciosas. Las heurísticas evalúan características estructurales de la URL, como su longitud, la presencia de una dirección IP en lugar de un dominio, el dominio de nivel superior o la edad del registro. Las basadas en aprendizaje automático representan la URL como un vector de características y la clasifican con un modelo estadístico.

Cada familia tiene compromisos distintos. Las listas negras son precisas pero tardan en incorporar amenazas nuevas; las heurísticas y los modelos de aprendizaje pueden detectar ataques que nadie ha reportado, a cambio de un mayor riesgo de falsos positivos. Esta diferencia justifica que el sistema completo esté organizado como una cascada: el filtro F1 consulta listas de reputación, F2.1 aplica heurísticas sobre la URL, F2.2 analiza el comportamiento del sitio en servicios externos y F2.3 obtiene una vista previa visual. En el alcance de este módulo se implementa F1 y se integran los resultados de los demás filtros, que desarrolla el módulo de servidor.

4.2. El estándar QR y su explotación como vector de ataque

4.2.1. Fundamentos técnicos del código QR

El código QR es una simbología bidimensional estandarizada por la norma ISO/IEC 18004 [1, 2]. Puede codificar hasta 4 296 caracteres alfanuméricos o 7 089 dígitos en una cuadrícula de módulos claros y oscuros. La especificación define cuatro modos de codificación (numérico, alfanumérico, byte y kanji), cuatro niveles de corrección de errores con redundancias de entre el 7 % y el 30 %, y cuarenta versiones de tamaño. La edición de 2024 mantiene la compatibilidad con la de 2015.

La corrección de errores explica parte de su éxito, porque permite leer un código aunque esté

parcialmente dañado. También explica una decisión de este trabajo: un código difuminado puede seguir siendo legible, por lo que ocultar un QR en una imagen exige cubrirlo por completo, como se describe en el capítulo de desarrollo.

4.2.2. Tipos de contenido codificable

Aunque la norma define la codificación a nivel de bytes, en la práctica los generadores siguen convenciones para tipos de contenido específicos: URLs, texto plano, contactos en formato vCard, credenciales de redes Wi-Fi, correos (`mailto:`), teléfonos (`tel:`) y coordenadas (`geo:`). Cada tipo tiene un prefijo reconocible en el texto decodificado.

No todos representan el mismo riesgo. Krombholz et al. identifican las URLs como el vector de mayor impacto y prevalencia [3]. Por esa razón, la unidad de análisis de este trabajo es exclusivamente la URL textual, según la sintaxis del RFC 3986 [15], y el módulo de escaneo necesita clasificar el contenido para decidir si corresponde el análisis de seguridad o una pantalla informativa.

4.2.3. El QR como vector de ataque

La superficie de ataque es amplia porque los códigos aparecen en entornos de alta confianza. Un atacante puede reemplazar un QR legítimo con una etiqueta adhesiva o distribuirlo en campañas de correo dirigidas. Palo Alto Networks Unit 42 documentó que las plataformas de correo no analizan automáticamente el contenido de los códigos incrustados en imágenes, lo que convierte al quishing en una técnica de evasión efectiva [6]. La aplicación interviene en el único punto del ciclo que todavía no tiene protección automatizada: el momento en que el usuario escanea con su teléfono.

4.3. Sistemas de detección y clasificación de amenazas

4.3.1. Listas negras y sistemas de reputación

Una lista negra de URLs es una base de datos de direcciones, dominios o IPs identificadas como maliciosas. Los sistemas de reputación extienden la idea asignando una puntuación a partir de varias señales, como el historial de actividad, la asociación con infraestructura conocida y las características del registro del dominio.

Sheng et al. encontraron en 2009 que las listas más grandes cubrían alrededor del 20 % de los sitios de phishing en el momento en que aparecían, y que la cobertura crecía conforme los sitios eran reportados [7]. Las plataformas actuales se actualizan en intervalos mucho más cortos, pero los informes recientes del APWG muestran que el phishing de vida corta sigue siendo una fracción importante de los ataques [8, 9]. Por lo tanto, la limitación de fondo se mantiene: una lista solo conoce lo que alguien ya reportó. Aun así, las listas son el primer filtro ideal, porque responden en milisegundos y tienen muy pocos falsos positivos.

4.3.2. Threat intelligence colaborativa

La *threat intelligence* es la disciplina que recolecta, analiza y comparte información sobre amenazas, incluidos los indicadores de compromiso (IOC) como dominios, URLs e IPs vinculados a campañas maliciosas. Las plataformas colaborativas parten de que ninguna organización ve el panorama completo por sí sola. MISP y URLhaus son ejemplos de código abierto [16, 17], y VirusTotal

agrega los resultados de más de setenta motores de análisis [18]. La función de reporte de QRisk sigue este principio a escala reducida: además de consumir inteligencia, el usuario puede aportar un caso nuevo.

4.3.3. Análisis heurístico de URLs

El análisis heurístico estima el riesgo de una URL a partir de su estructura, sin consultar bases externas ni visitar el sitio. Las señales más usadas son la longitud anómala, la IP literal como host, los subdominios engañosos, los dominios de nivel superior poco convencionales, los acortadores, la ausencia de HTTPS y las palabras clave asociadas a credenciales. Ma et al. y Le et al. mostraron que las características léxicas de la URL permiten clasificar con alta precisión sin acceder al contenido [19, 20]. El filtro F2.1 del backend aplica este enfoque y devuelve un puntaje de 0 a 100 con los criterios que lo componen; la aplicación los presenta tal cual en la vista de detalle.

4.3.4. Aprendizaje automático aplicado a URLs

Los enfoques de aprendizaje automático formulan la detección como una clasificación binaria o multiclase sobre características léxicas, de registro, de DNS o de contenido, con algoritmos como bosques aleatorios, *gradient boosting* o redes neuronales [14, 19]. Este trabajo no entrena modelos propios, pero el concepto es relevante por dos razones: el filtro heurístico podría evolucionar hacia un modelo entrenado, y los conjuntos de datos etiquetados que se usan en esta literatura sirven también para evaluar sistemas de filtrado. Uno de ellos, el conjunto público de URLs maliciosas de Kaggle [21], se utilizó en las pruebas de este trabajo.

4.3.5. Métricas de evaluación de clasificadores

Evaluar un clasificador binario requiere más que la exactitud, sobre todo cuando las clases están desbalanceadas. Las métricas habituales son la precisión, la exhaustividad (*recall*), la especificidad y la puntuación F1 [22]. En seguridad los errores no cuestan lo mismo: un falso negativo deja pasar una amenaza y un falso positivo bloquea un sitio legítimo. En QRisk esa asimetría se traduce en una regla de diseño: un falso "seguro" es el peor resultado posible, por lo que, ante información incompleta, la aplicación prefiere comunicar "incierto".

4.4. Escaneo y procesamiento nativo en dispositivos móviles

4.4.1. APIs nativas de detección de códigos

Los sistemas operativos móviles ofrecen APIs para detectar y decodificar códigos directamente en el dispositivo, sin servicios en la nube. En iOS, el framework Vision ofrece la solicitud `VNDetectBarcodesRequest` [23], además de la detección de códigos legibles por máquina de AVFoundation [24]. En Android, Google ML Kit ofrece el escaneo de códigos de barras con funcionamiento local y sin conexión [25]. Procesar en el dispositivo tiene dos ventajas para este trabajo: la imagen nunca sale del teléfono y la decodificación ocurre en fracciones de segundo, dejando el presupuesto de tiempo para el análisis de red.

4.4.2. Librerías de cámara para React Native

El protocolo especificaba `react-native-vision-camera`, una librería de código abierto que expone la cámara con procesamiento de cuadros mediante JSI y un módulo de lectura de códigos construido sobre las APIs nativas [26]. Su principal ventaja es el rendimiento en el procesamiento de video en tiempo real.

La alternativa es `expo-camera`, el módulo de cámara mantenido por el propio ecosistema de Expo [27]. Ofrece vista previa de cámara, control de linterna, gestión de permisos y lectura de códigos de barras restringible a formatos específicos, como QR. Su ventaja es que se publica alineado con cada versión del SDK de Expo, lo que reduce los conflictos entre dependencias nativas. Como se explica en el capítulo de desarrollo, esta fue la librería finalmente adoptada.

4.4.3. Latencia de decodificación

La latencia de decodificación es el intervalo entre el momento en que el código entra en el campo de visión de la cámara y el momento en que la aplicación recibe su contenido. Depende de la resolución del cuadro analizado, la densidad del código, la iluminación, el ángulo, la capacidad del dispositivo y la eficiencia del decodificador. Es el primer componente del tiempo total que percibe el usuario, al que luego se suman la consulta de reputación y la cascada del servidor. Por eso el protocolo fija un criterio de 5 s en el percentil 90 solo para la lectura.

4.5. Desarrollo móvil multiplataforma con React Native

4.5.1. Arquitectura de React Native

React Native permite escribir la interfaz en JavaScript o TypeScript y renderizarla con componentes nativos de cada plataforma. Su arquitectura histórica usaba un puente asíncrono basado en la serialización de mensajes, lo que introducía latencia en operaciones frecuentes. La llamada *New Architecture* reemplaza ese puente por tres componentes: JSI, que permite llamadas síncronas entre JavaScript y código nativo; Fabric, el nuevo motor de renderizado; y TurboModules, que cargan los módulos nativos bajo demanda [28]. El prototipo se construyó sobre React Native 0.86 con esta arquitectura activa.

4.5.2. Expo y el desarrollo desde una sola base de código

Expo es un conjunto de herramientas sobre React Native que agrega módulos nativos mantenidos, enrutamiento basado en archivos mediante Expo Router [29] y compilación en la nube mediante EAS Build [30]. Con el enrutamiento por archivos, cada pantalla de la aplicación corresponde a un archivo dentro de una carpeta, lo que hace que la estructura de navegación se lea directamente en el árbol del proyecto.

Desarrollar desde una sola base de código es lo que hizo viable que una sola persona construyera el módulo. Sin embargo, las políticas de permisos, almacenamiento y red difieren entre iOS y Android, y esas diferencias deben resolverse explícitamente en la configuración de cada plataforma.

4.6. Seguridad en aplicaciones móviles

4.6.1. OWASP MASVS

El *Mobile Application Security Verification Standard* de OWASP define los requisitos de seguridad para aplicaciones móviles. Su versión 2.1 organiza los controles en categorías como almacenamiento, criptografía, autenticación, red y plataforma [31]. Una aplicación que pretende proteger al usuario de amenazas externas debe ser segura en sí misma. Para el prototipo aplican tres categorías: almacenamiento, por el historial local; red, por la comunicación con los servicios de análisis; y plataforma, por el manejo de los permisos de cámara y ubicación.

4.6.2. Seguridad de red en iOS y Android

En iOS, *App Transport Security* exige por defecto conexiones HTTPS con TLS 1.2 o superior, y cualquier excepción debe declararse en la configuración de la aplicación [32, 33]. En Android, la *Network Security Configuration* cumple una función equivalente, y desde Android 9 el tráfico sin cifrar está deshabilitado por defecto [34]. Todos los servicios que consume QRisk usan HTTPS. La URL escaneada, que puede ser HTTP, nunca se abre dentro de la aplicación: se envía como texto a los servicios de análisis, por lo que no hace falta declarar excepciones.

4.6.3. Almacenamiento local

El historial de escaneos debe estar disponible sin conexión y sin cuenta de usuario. El protocolo proponía `react-native-mmkv`, una librería clave valor de lectura síncrona y alto rendimiento [35, 36]. El prototipo utiliza el almacén clave valor de `expo-sqlite` [37], que también ofrece una interfaz síncrona de lectura. Esa propiedad es la que importa para la experiencia de uso, porque permite que la aplicación decida qué mostrar desde el primer renderizado. La diferencia en materia de cifrado se discute en el capítulo de desarrollo.

4.7. Fuentes de reputación del filtro F1

4.7.1. Google Safe Browsing

Google Safe Browsing es el servicio que permite consultar si una URL aparece en las listas de amenazas que mantiene Google, que cubren phishing, malware y software no deseado. Es la misma fuente que utilizan Chrome, Safari y Firefox para mostrar advertencias [10]. Su método de búsqueda de coincidencias recibe la URL completa y responde si existe alguna amenaza asociada, lo que lo hace adecuado para consultas puntuales desde un cliente. Al operar sobre la URL completa y no solo sobre el dominio, puede conocer una página maliciosa alojada en un dominio legítimo.

4.7.2. PhishTank

PhishTank es una base colaborativa de URLs de phishing construida a partir de reportes verificados por la comunidad y operada por Cisco Talos [11]. Ofrece un servicio de consulta por URL que responde si la dirección está registrada y si fue verificada. Desde finales de 2023 su mantenimiento se ha reducido y el servicio presenta interrupciones y límites de consulta no documentados, por lo que

cualquier integración debe tratar su indisponibilidad como un caso normal y no como una excepción. OpenPhish es una alternativa activa que puede funcionar como fuente complementaria [38].

4.7.3. VirusTotal

VirusTotal agrega los resultados de más de setenta motores antivirus y escáneres de URL, y su API permite recuperar el veredicto de cada uno [18]. Su fortaleza es la agregación: una URL que evade un motor puede ser detectada por otros. Su limitación para este proyecto es la cuota del plan gratuito, de 4 consultas por minuto y 500 por día. En el sistema final, VirusTotal se consulta dentro del filtro F2.2 del servidor, y la aplicación presenta el recuento de motores que marcan la URL.

4.7.4. URLhaus y MISP

URLhaus, de abuse.ch, se especializa en URLs que distribuyen malware, a diferencia de PhishTank, que se centra en phishing [17]. MISP es una plataforma de código abierto para intercambiar indicadores de compromiso entre comunidades [16]. Ninguna de las dos se integra en el prototipo, pero ambas son relevantes: URLhaus se usó como fuente de amenazas activas en las pruebas, y MISP representa el modelo hacia el que podría evolucionar la función de reporte.

4.8. Geolocalización y geocodificación inversa

Los dispositivos determinan su ubicación combinando GPS, redes Wi-Fi y torres celulares, gestionados por Core Location en iOS y por los servicios de ubicación de Android [39, 40]. Ambas plataformas exigen el consentimiento explícito del usuario y distinguen entre uso en primer plano y en segundo plano.

La geocodificación inversa convierte coordenadas en una descripción legible, como el nombre de un lugar y la ciudad. Puede resolverse con los servicios del sistema operativo, como `CLGeocoder` en iOS [41], o con APIs de terceros como Mapbox [42]. La elección tiene implicaciones de privacidad, porque en el segundo caso las coordenadas del usuario viajan a un servidor externo. QRisk resuelve la geocodificación con el servicio del sistema mediante `expo-location` y no envía las coordenadas a su propio backend.

4.9. Usabilidad y seguridad centrada en el usuario

4.9.1. Usable security

La *usable security* estudia cómo diseñar sistemas de seguridad que las personas puedan usar de forma efectiva. Cranor propuso un marco según el cual el comportamiento seguro requiere tres pasos: que el usuario note la señal, que entienda su significado y que actúe en consecuencia [43]. Un sistema que falla en cualquiera de los tres es inefectivo, sin importar qué tan sólido sea técnicamente. Si QRisk presenta un veredicto confuso, tardío o demasiado técnico, el usuario puede ignorarlo, y el beneficio de la herramienta desaparece.

4.9.2. Efectividad de las advertencias

Egelman, Cranor y Hong encontraron que las advertencias de phishing más efectivas interrumpen la tarea, son visualmente prominentes, usan lenguaje concreto y aparecen en el momento de mayor relevancia [44]. Akhawe y Felt analizaron millones de advertencias reales en Chrome y Firefox, y observaron que la proporción de usuarios que continúa a pesar de la advertencia varía mucho según el tipo: cerca del 70 % en las advertencias de certificados de Chrome y mucho menos en las de malware y phishing [45]. Es decir, la severidad percibida y la claridad del mensaje cambian la conducta.

Estos hallazgos guían las pantallas de resultado de QRisk: el veredicto en palabras simples y con color, la presentación en dos niveles de detalle para no saturar al usuario promedio, y una fricción que aumenta con el riesgo cuando la persona decide abrir el sitio.

4.10. Arquitectura de backend en la nube

El motor de análisis se despliega con un modelo sin servidor sobre AWS Lambda, con Amazon API Gateway como punto de entrada HTTP [46]. En este modelo el proveedor asigna recursos por solicitud, escala de forma automática y no cobra cuando no hay ejecución. Tiene dos consecuencias que afectan directamente a la aplicación. La primera es el arranque en frío: cuando una función no ha recibido solicitudes durante un tiempo, la primera petición debe inicializar el entorno y tarda más. La segunda es que API Gateway impone un tiempo máximo por solicitud, de modo que un análisis que lo supera se corta con un error aunque la función siga trabajando. Ambos efectos se observaron en las mediciones y explican varias decisiones de la política de tiempos descrita en el capítulo de desarrollo.

CAPÍTULO 5

Antecedentes

El problema de este trabajo cruza cuatro líneas de investigación que han avanzado bastante por separado: el estudio del código QR como vector de ataque, la detección automática de URLs maliciosas, los sistemas de reputación y las plataformas colaborativas de *threat intelligence*, y la *usable security*, que estudia cuándo una advertencia cambia realmente lo que hace la persona que la recibe. Al revisarlas juntas aparece un desbalance. Las tres primeras tienen resultados sólidos y medibles; la cuarta ha mostrado una y otra vez que detectar bien no es suficiente para proteger. Este capítulo revisa los trabajos de cada línea, compara sus enfoques y delimita el vacío que el prototipo busca cubrir.

5.1. Procedimiento de la revisión

La búsqueda se realizó en IEEE Xplore, ACM Digital Library, SpringerLink, arXiv y Google Scholar, además de la documentación oficial de los proveedores de reputación y de las plataformas móviles, y de informes públicos de la industria de seguridad. Se combinaron términos como *QR code phishing*, *quishing*, *malicious URL detection*, *phishing blacklist coverage*, *security warning effectiveness* y *URL reputation*, en inglés y en español, con un rango principal de 2008 a 2025. Se incluyeron trabajos fundacionales anteriores cuando siguen siendo referencia.

Se dejaron fuera los trabajos sobre el procesamiento de imagen del símbolo QR sin componente de seguridad y las propuestas de firmar los códigos con esquemas criptográficos, porque obligan a cambiar el estándar o la infraestructura de quien emite el código. No se trata de una revisión sistemática formal: no hubo doble revisión independiente ni protocolo registrado. Por lo tanto, cuando este capítulo afirma que algo no se encontró, debe leerse como que no apareció con la búsqueda descrita, y no como que no existe.

5.2. Revisión de antecedentes

5.2.1. El código QR como vector de ataque

El trabajo que abrió esta línea es el de Krombholz et al., la primera taxonomía ordenada de ataques sobre códigos QR desde la perspectiva de la usabilidad [3]. Su aporte principal fue separar los ataques que manipulan el código físico, como pegar una calcomanía encima, de los que aprovechan el contenido codificado. Identificaron las URLs como el contenido más peligroso y más común, y plantearon el problema como un asunto de usabilidad: el usuario no puede verificar lo que no puede leer.

La evidencia empírica más citada viene de Vidas et al., quienes colocaron códigos QR en lugares públicos y midieron cuántas personas los escaneaban [4]. Encontraron que la mayoría escaneaba por curiosidad, sin revisar el destino, y que la susceptibilidad era mayor que en el phishing por enlace directo. Su valor metodológico está en que midieron conducta real y no lo que la gente decía que haría.

Ambos trabajos son de 2013 y 2014, antes de que el uso de códigos QR se masificara. En ese momento escanear requería instalar una aplicación aparte; hoy la cámara del teléfono lo hace sola, lo que elimina todavía más fricción. Sus conclusiones probablemente siguen vigentes, pero sus cifras deben leerse como un piso y no como una estimación actual. Para conocer el estado reciente del fenómeno hay que recurrir a los informes de la industria: Insikt Group documentó la integración del quishing a plataformas de phishing como servicio [5], y Unit 42 explicó el incentivo, que es la incapacidad de los filtros de correo para leer un QR incrustado como imagen [6].

5.2.2. Detección de URLs maliciosas

Sahoo, Liu y Hoi organizan las técnicas en listas negras, heurísticas y aprendizaje automático, y explican qué gana y qué pierde cada una [14]. Sheng et al. aportaron la evaluación más citada de los límites de las listas negras: alrededor del 20 % de cobertura en el momento en que aparecía un sitio de phishing, con una mejora progresiva conforme el sitio era reportado [7]. Esa cifra es de 2009 y no puede trasladarse al presente, porque las plataformas actuales se actualizan mucho más rápido. Lo que sí sigue vigente es la limitación de fondo: ninguna lista puede contener algo que todavía no se ha visto, y los informes del APWG muestran que el phishing de vida corta sigue siendo una parte importante de los ataques [8, 9].

Frente a ese límite, los clasificadores basados en características de la URL son especialmente prometedores para un contexto móvil. Ma et al. mostraron que con información del dominio y de la estructura léxica se alcanzan precisiones cercanas al 99 % sin visitar el sitio [19], y Le et al. llevaron la idea al extremo con PhishDef, que usa únicamente la URL [20]. No tener que abrir el sitio tiene además una ventaja de privacidad que la literatura menciona poco: nadie se entera de que un usuario concreto está por visitar una URL concreta. Sus límites son que los resultados provienen de conjuntos históricos en condiciones de laboratorio y que las técnicas de evasión, como los acortadores o los dominios legítimos comprometidos, reducen su efectividad. Por eso, en el sistema propuesto, la heurística es un filtro posterior y no el único mecanismo.

5.2.3. Sistemas de reputación y threat intelligence

La infraestructura en la que se apoya el filtro F1 está documentada sobre todo en manuales técnicos oficiales y no en publicaciones arbitradas, lo cual ya dice algo sobre el estado del arte. Google Safe Browsing es el sistema con mayor alcance y el que alimenta las advertencias de los

principales navegadores [10]; su desventaja es que no permite auditar por qué marca o no marca una URL. VirusTotal junta los veredictos de decenas de motores, lo que reduce los falsos negativos, pero su cuota gratuita limita mucho la validación empírica [18]. PhishTank y URLhaus funcionan de forma colaborativa [11, 17], y su cobertura depende de qué tan activa sea la comunidad, lo que probablemente la vuelve más débil para campañas en español dirigidas a la región.

En la literatura revisada no se encontró un estudio independiente que compare qué tanto coinciden estas fuentes sobre un mismo conjunto de URLs, ni cómo se comporta un veredicto combinado frente a cada fuente por separado. Cada proveedor documenta lo que él hace, pero nadie documenta cómo funcionan juntos. Las pruebas de este trabajo aportan, de forma modesta, en esa dirección.

5.2.4. Soluciones comerciales de escaneo seguro

Aplicaciones como *Safe QR Scan* y *SafeQR Reader* revisan la URL antes de redirigir comparándola con bases de dominios maliciosos. Su mérito es haber mostrado que el punto de intervención funciona y que los usuarios aceptan ese paso intermedio. Sus limitaciones son tres: dependen casi exclusivamente de listas negras, no publican su arquitectura ni sus tasas de error, y no explican en qué basaron sus decisiones de interfaz. Este trabajo no incluye una comparación formal contra ellas, exclusión declarada en el capítulo de alcance; se mencionan para describir la práctica actual.

5.2.5. Usable security y efectividad de advertencias

Cranor propuso descomponer el comportamiento seguro en notar la señal, entenderla y actuar en consecuencia [43]. Egelman, Cranor y Hong encontraron que las advertencias más efectivas interrumpen la tarea, son prominentes y usan lenguaje concreto [44]. Akhawe y Felt mostraron con datos reales a gran escala que la proporción de usuarios que ignora una advertencia depende del tipo de amenaza y de cómo se presenta [45]. El problema común es el contexto: los tres trabajos estudian navegadores de escritorio, y no se encontró una validación de sus hallazgos en el entorno móvil, con pantalla pequeña, interacción táctil y uso apresurado. Este trabajo asume esa extrapolación de forma explícita.

5.3. Comparación de los trabajos revisados

El cuadro 5.1 ordena los antecedentes según el momento del ataque en que intervienen y el tipo de evidencia que aportan.

Tabla 5.1: Comparación de enfoques según su punto de intervención

Enfoque	Dónde actúa	Fortaleza	Limitación
Listas de reputación [10, 11, 18]	Sobre URLs ya conocidas	Rápidas y precisas	No ven amenazas nuevas
Clasificadores léxicos [19, 20]	Sobre la estructura de la URL	Evalúan dominios nuevos sin visitarlos	Más falsos positivos; evaluados en laboratorio
Filtros de correo [6]	Antes de que llegue el mensaje	Protegen sin intervención del usuario	No leen el QR
Advertencias del navegador [44, 45]	Después de abrir la URL	Evidencia sólida sobre comunicación	Llegan cuando ya hubo contacto con el atacante
Aplicaciones comerciales	Entre el escaneo y la apertura	Punto de intervención correcto	Detección limitada a listas; sin documentación

De esta comparación sale el hallazgo central del capítulo: el punto donde la intervención es posible coincide con el punto donde ningún trabajo revisado combina detección en varias capas con una comunicación fundamentada del veredicto. El intervalo entre leer el código y abrir el navegador es el único en que la URL ya es visible, el ataque todavía no se consuma y el usuario está en posición de decidir.

5.4. Limitaciones identificadas en la literatura

La evidencia empírica sobre la susceptibilidad frente a códigos QR es anterior a su masificación. La evidencia sobre advertencias proviene de navegadores de escritorio y no se ha validado en móviles. Los clasificadores reportan su desempeño en laboratorio y no en un teléfono, con sus restricciones de cómputo y de red. No hay evaluaciones independientes de las fuentes de reputación operando juntas. Las soluciones comerciales son opacas. Y, sobre todo, la detección y la comunicación se han estudiado como problemas separados, cuando para el usuario final son una sola experiencia.

5.5. Fundamentación del proyecto

Cada decisión del prototipo responde a una de estas limitaciones. El punto de intervención en la aplicación de escaneo responde a que es el único lugar donde la URL es legible antes del contacto con el atacante. La arquitectura en cascada responde a que ninguna técnica individual cubre todas las amenazas: las listas resuelven rápido lo conocido y los filtros posteriores evalúan lo nuevo. Las decisiones de interfaz, como el veredicto en lenguaje simple, los dos niveles de detalle y la fricción proporcional al riesgo, aplican la evidencia de usable security al contexto móvil. Y la regla de nunca visitar la URL desde el teléfono aprovecha la ventaja de privacidad del análisis sin contacto.

Por lo tanto, el aporte del trabajo no está en inventar una técnica de detección nueva, sino en llevar capacidades existentes al momento y al dispositivo donde ocurre el ataque, documentando de forma abierta la arquitectura, las fuentes y las decisiones de comunicación, y evaluándolas con criterios medibles.

CAPÍTULO 6

Alcance

6.1. Definición del alcance

Este trabajo comprende el diseño, la implementación y la evaluación del módulo móvil de QRisk, que forma parte de un sistema compuesto por una aplicación en React Native y un motor de análisis desplegado en AWS. El módulo cubre el flujo completo de la experiencia de usuario y se organiza en tres bloques:

- **Escaneo e interacción principal:** lectura del código QR con la cámara, clasificación del contenido, pantalla de carga y presentación del veredicto en dos niveles de detalle.
- **Filtro F1 de reputación:** resolución en el dispositivo del primer filtro del sistema y su integración de extremo a extremo con la cascada del servidor. La aplicación también presenta los resultados de los filtros F2.1 (heurístico), F2.2 (comportamiento) y F2.3 (vista previa visual), cuya implementación corresponde al módulo de servidor.
- **Funcionalidades secundarias:** historial local disponible sin conexión con un máximo de 20 escaneos, reporte anónimo de códigos maliciosos y la Story Card compartible con geocodificación inversa.

El producto final es un prototipo instalable en dispositivos Android físicos. No contempla su publicación en tiendas de aplicaciones.

6.2. Delimitación funcional y técnica

Tabla 6.1: Componentes incluidos en el módulo

Componente	Descripción
Escaneo	Decodificación del QR en tiempo real con los servicios nativos del dispositivo, linterna y bloqueo tras la primera lectura
Clasificación	Separación entre URL y contenido no navegable (Wi-Fi, contacto, correo, teléfono, ubicación y texto)
Filtro F1	Lista local de dominios confiables y consultas en paralelo a Google Safe Browsing y PhishTank
Integración con la cascada	Envío de la URL y del resultado de F1 al servidor, lectura de la respuesta y presentación de los filtros F2.1, F2.2 y F2.3
Resultado	Pantalla de carga por etapas, resultado simple y resultado extendido
Historial	Últimos 20 escaneos, consultables sin conexión y sin cuenta de usuario
Reporte	Envío anónimo de un código sospechoso al servidor
Story Card	Imagen compartible con foto del entorno, QR cubierto, lugar y veredicto

La unidad de análisis es exclusivamente la URL textual extraída del código, según el RFC 3986 [15]. No se analiza el símbolo QR como imagen ni el resto de los contenidos codificables, que solo se muestran al usuario de forma segura.

6.3. Restricciones

- **Temporal:** diecisiete semanas de trabajo efectivo dentro de un semestre de veinte, con un margen para imprevistos.
- **Equipo:** dos integrantes, cada una responsable de un módulo, sin recursos adicionales.
- **Económica:** el sistema opera dentro de los planes gratuitos de las APIs externas y de la capa gratuita de AWS.
- **Hardware:** las pruebas se realizan sobre un teléfono Android de gama media. No se dispuso de una computadora con macOS para compilar la versión de iOS.
- **Conectividad:** el análisis requiere conexión a internet. Sin conexión, la aplicación no produce un veredicto, lo comunica al usuario y mantiene disponible el historial.

6.4. Exclusiones explícitas

- **Funcionales:** cuentas de usuario y autenticación, historial ilimitado, animaciones del comportamiento del sitio, notificaciones y escaneo en segundo plano.
- **De contenido:** análisis de seguridad sobre credenciales Wi-Fi, contactos o esquemas de pago codificados en el QR, y URLs con esquemas distintos de `http` y `https`.

- **Técnicas:** implementación de los filtros F2.1, F2.2 y F2.3; modelos propios de aprendizaje automático; análisis de HTML o JavaScript en el dispositivo; publicación en tiendas; pruebas de carga a escala.
- **De plataforma:** la versión para iOS. Aunque el protocolo contemplaba ambas plataformas, la entrega se limita a Android por la falta de hardware de Apple durante el desarrollo. La configuración que no requiere macOS se conserva en el proyecto para que pueda retomarse.
- **De investigación:** estudios de usabilidad con muestras estadísticamente representativas, estudios de campo en entornos reales y comparaciones con soluciones comerciales.

CAPÍTULO 7

Metodología

7.1. Enfoque del trabajo

El presente trabajo es un proyecto de desarrollo de software de naturaleza aplicada. Su objetivo no es generar conocimiento teórico nuevo sobre la detección de URLs maliciosas, tema ya tratado en los antecedentes, sino construir un artefacto funcional que lleve capacidades de análisis existentes al único punto de la cadena del ataque donde todavía pueden aplicarse: el intervalo entre la lectura del código QR y la apertura del navegador. El desarrollo se enmarca en la modalidad de prototipado, apropiada cuando los requisitos se refinan a medida que se validan los componentes del sistema.

La evaluación se organiza alrededor de dos preguntas verificables. La primera es si el prototipo cumple los tiempos y las tasas de acierto que lo vuelven usable. La segunda es si el veredicto que produce se entiende lo suficiente como para que el usuario tome una buena decisión.

Desde el inicio se estableció una distinción que ordena todo el diseño de la evaluación. Este trabajo mide el desempeño del módulo móvil: la latencia del escaneo, la tasa de lectura de códigos, la clasificación del contenido, el tiempo del filtro F1, la integridad del flujo y la comprensión del veredicto. La capacidad de detección de los filtros F2.1, F2.2 y F2.3 depende del módulo de servidor. Cuando una métrica depende de ese módulo, se declara explícitamente.

7.2. Metodología de gestión

Para organizar el trabajo se adoptó Kanban, un método de gestión visual que representa las tareas como tarjetas que avanzan por un tablero de estados [47]. El tablero se llevó en Monday.com con las columnas pendiente, en progreso, en revisión y completado, y se dividió en dos carriles, uno por módulo, con un espacio para las tareas de integración que requerían coordinación entre ambas integrantes. Una tarea pasaba a completada solo cuando la funcionalidad estaba integrada y verificada con pruebas o contra el contrato de interfaz.

La elección respondió a las características del equipo y del problema. Con dos integrantes y res-

ponsabilidades diferenciadas, una metodología con roles y ceremonias como Scrum resultaba más pesada de lo necesario. Una metodología secuencial tampoco era adecuada, porque el comportamiento real de varias piezas, como las APIs de reputación, las librerías de cámara y la latencia del servidor, solo podía conocerse al implementarlas. Kanban permitió reordenar prioridades sin esperar el cierre de un ciclo, que fue justamente lo que ocurrió durante el proyecto.

7.3. Fases del proyecto

El protocolo organizó el trabajo en seis fases a lo largo de diecisiete semanas, resumidas en el cuadro 7.1.

Tabla 7.1: Fases planificadas en el protocolo

Fase	**Semanas**	**Entregable principal**
1. Diseño y configuración del entorno	1 y 2	Contrato de interfaz inicial y wireframes del flujo principal
2. Escaneo, F1 y arquitectura base	3 a 6	Flujo de extremo a extremo: escaneo, F1 y resultado simple
3. Resultado extendido (F2.1)	7 a 9	Desglose heurístico en pantalla
4. Pantallas F2.2 y F2.3 y funcionalidades secundarias	10 a 12	Prototipo completo con historial, reporte y Story Card
5. Pruebas, refinamiento y documentación	13 a 15	Mediciones e informe del módulo
Cierre	16 y 17	Revisión del asesor y defensa

7.3.1. Reorganización por bloques de trabajo

A mitad del proyecto el orden de las fases dejó de describir el estado real del trabajo. Había tareas de fases avanzadas terminadas, como las pantallas de resultado construidas sobre datos de prueba, y tareas de fases iniciales todavía abiertas, como el escaneo en hardware y la integración con el servidor. Por esa razón el desarrollo se reorganizó en bloques ordenados por dependencia técnica y por riesgo, con tres reglas: primero lo que desbloquea el trabajo de la otra integrante, después lo que elimina más riesgo técnico y al final lo que solo depende del tiempo disponible. El cuadro 7.2 resume los bloques.

Tabla 7.2: Bloques de desarrollo y criterio de cierre

N.º	Objetivo	Criterio de cierre
0	Desbloquear el módulo de servidor	Contrato de interfaz entregado
1	Escaneo real en dispositivo	Prueba de lectura superada; decisión sobre la librería de cámara
2	Persistencia local	Historial y preferencias sobreviven al reinicio
3	Integración con el servidor	Flujo de extremo a extremo con degradación controlada
4	Funcionalidades parciales	Reporte, Story Card e historial operativos
5	Compatibilidad con iOS	Fuera de alcance por falta de hardware
6	Pruebas y métricas	Cobertura, mediciones y pruebas de usabilidad
7	Cierre y documentación	Informe y defensa

El bloque 5 se mantuvo en el plan aunque quedó fuera del alcance, para que la exclusión fuera una decisión visible y no una omisión.

7.3.2. El contrato de interfaz como instrumento

La coordinación con el módulo de servidor se formalizó mediante un contrato de interfaz escrito antes de que existiera el servicio. El contrato no se derivó de lo que el servidor podría ofrecer, sino de lo que la aplicación necesitaba: las pantallas ya construidas sobre datos de prueba determinaban exactamente qué campos hacían falta, con qué tipos y en qué momento. De esta forma, las definiciones de tipos de la aplicación funcionaron como una especificación ya validada contra una interfaz real.

El contrato fijó el formato JSON en UTF-8, las marcas de tiempo en ISO 8601 y la ausencia de cuentas de usuario, y trasladó a la API la regla de seguridad del producto: ningún servicio puede devolver HTML para que el cliente lo renderice ni redirigir el dispositivo hacia el sitio analizado. La versión vigente define una sola llamada bloqueante para el análisis y un servicio separado para los reportes, y se resume en el anexo B.

7.4. Estrategia de validación

La validación se organizó en tres niveles, separados por el tipo de pregunta que cada uno puede responder. Asignar a un nivel una pregunta que no puede contestar es la forma más común de producir evidencia que parece sólida sin serlo.

7.4.1. Nivel 1: pruebas automatizadas de la lógica de negocio

El primer nivel cubre la capa de servicios mediante pruebas unitarias con Jest y el preset de Expo. La cobertura se mide únicamente sobre esa capa, con un umbral del 70 % en sentencias, ramas, funciones y líneas que interrumpe la ejecución si no se alcanza. Las pantallas y el sistema de diseño quedan fuera a propósito: que un color comunique riesgo o que la jerarquía de botones lleve a la decisión correcta no es algo que una prueba unitaria pueda afirmar, y se valida en el dispositivo y con usuarios.

El aislamiento se logra sustituyendo el almacén clave valor por una implementación en memoria con la misma interfaz síncrona, simulando la función de red y reemplazando el cliente HTTP y el módulo de ubicación. Ninguna prueba realiza peticiones reales, por lo que el conjunto es determinista y reproducible sin conexión ni credenciales. Algunas pruebas fijan además decisiones de diseño que podrían romperse en silencio, como el valor de los tiempos límite o la tabla de códigos QR de ejemplo de la documentación, que se recorre completa para verificar que cada entrada produzca la clasificación que declara.

7.4.2. Nivel 2: verificación en dispositivo y contra servicios reales

El segundo nivel cubre lo que no puede verificarse desde una computadora: la cámara, los diálogos del sistema operativo, el GPS, la galería, la acción de compartir y el comportamiento real de los servicios externos. Comprende el banco de pruebas de lectura, el protocolo de verificación en el teléfono y las corridas del código real de la aplicación contra los servicios de reputación y contra el servidor, descritos en la sección 7.5.

7.4.3. Nivel 3: pruebas de usabilidad

El tercer nivel evalúa lo que ninguna prueba automatizada responde: si el veredicto se entiende. Se planificó con cinco participantes y tres indicadores: comprensión del veredicto, con criterio de al menos el 90 %; finalización del flujo sin ayuda, con criterio de al menos el 85 %; y tiempo del primer escaneo exitoso, con criterio de 60 s como máximo. Medir la comprensión por separado del desempeño técnico se basa en el modelo de Cranor, según el cual un comportamiento seguro exige notar, entender y actuar [43]. Un veredicto correcto que no se entiende no cambia la conducta y, por lo tanto, no protege.

El número de participantes sigue el argumento de Nielsen y Landauer de que una muestra pequeña ya permite encontrar la mayor parte de los problemas de usabilidad de una interfaz [48]. Con cinco personas los resultados se interpretan como indicios y no como estimaciones poblacionales.

7.5. Diseño de las mediciones

7.5.1. Tiempo de lectura y tasa de detección

La lectura del código se evalúa con tres indicadores. La tasa de detección (TDR) es el número de códigos leídos dividido entre el número de códigos presentados, con criterio de al menos el 95 %. El tiempo de escaneo (T_{scan}) es el intervalo entre el momento en que el lector está listo y el momento en que entrega el contenido, con criterio de 5 s o menos en el percentil 90; no incluye el análisis posterior, que se mide aparte. La precisión de clasificación es la proporción de códigos que abren la pantalla correcta según su contenido, con criterio de al menos el 98 %.

Se eligió el percentil 90 y no el promedio porque una sola lectura lenta no debe descalificar la aplicación, pero tampoco debe poder esconderse en un promedio. Con nueve lecturas de 500 ms y una de 20 s el percentil 90 sigue cumpliendo el criterio; hacen falta dos lecturas lentas para moverlo. El percentil se calcula por interpolación lineal [49].

La aplicación ya incluye la instrumentación para medir en el teléfono: cada lectura registra su duración y un resumen calcula la mediana, el percentil 90, el máximo y la tasa de detección. Esta forma de medir tiene una limitación: los fallos deben anotarse manualmente, porque la aplicación no tiene manera de saber que hubo un código frente a la cámara que nunca se decodificó.

Banco de pruebas automatizado de lectura

Para reducir la dependencia del cronometraje manual se planificó un banco de pruebas automatizado. En lugar de presentar cada código frente a la cámara y anotar el resultado, el banco toma un conjunto de imágenes de códigos QR almacenadas en una carpeta, las procesa una por una con el lector y registra automáticamente el tiempo de cada lectura y si fue exitosa o no. De esta forma los fallos quedan registrados sin intervención humana y la medición puede repetirse las veces que sea necesario con exactamente las mismas imágenes.

Pendiente: Describir el banco de pruebas cuando esté construido: herramienta y lector utilizados, cómo se generaron o fotografiaron las imágenes, cantidad de imágenes por condición (tamaño, ángulo, iluminación y tipo de contenido), cómo se mide el tiempo, en qué equipo corre y qué diferencia hay entre decodificar una imagen fija y leer desde la cámara en vivo.

La muestra de referencia del protocolo varía las tres condiciones que impone el uso real, según el cuadro 7.3. El banco automatizado debe conservar esta misma variedad para que sus resultados sean comparables con el criterio del protocolo.

Tabla 7.3: Composición de referencia de la muestra de lectura

Condición	**Niveles**
Tamaño	Pequeño (cerca de 2 cm, etiqueta o ticket), mediano (cerca de 5 cm, volante o menú) y grande (15 cm o más, cartel a distancia)
Ángulo	De frente, inclinado cerca de 30° e inclinado cerca de 45°
Iluminación	Luz de día, penumbra y oscuridad con linterna
Contenido	Mayoría con URL; el resto con credenciales Wi-Fi, texto, contacto, correo, teléfono y ubicación

El protocolo estableció también un criterio de decisión sobre la librería de cámara. Si la tasa de detección resultara inferior al 95 % o el percentil 90 superara los 5 s, correspondería migrar a `react-native-vision-camera`. Antes de concluirlo se descartan tres causas de menor costo: que el fallo se concentre en ángulos de 45°, en penumbra o en códigos pequeños a distancia. Si el fallo se concentra en un solo eje se documenta como una limitación conocida; si se reparte en los tres, la migración queda justificada.

7.5.2. Mediciones de los filtros de análisis

El tiempo del filtro F1 (T_{F1}) se mide desde que la aplicación inicia las consultas de reputación hasta que obtiene su conclusión, con criterio de 5 s en el percentil 90. El tiempo total (T_{total}) se mide desde el envío de la URL hasta que la aplicación tiene el veredicto final, con criterio de 35 s para la cascada completa.

Para estas mediciones se ejecutó el código real de la aplicación, y no una réplica, sobre muestras públicas de URLs. Se usaron dos fuentes. La primera fue el conjunto de URLs maliciosas publicado en Kaggle [21], del que se extrajeron muestras aleatorias reproducibles con semilla fija, equilibradas entre las clases phishing, malware, desfiguración y benignas. La segunda fue el registro reciente de URLhaus [17], que contiene amenazas activas reportadas pocas horas antes de la consulta. Estas muestras permiten medir tiempos y observar qué filtro detecta cada tipo de amenaza, pero tienen limitaciones de etiquetado que se discuten en los resultados.

La tasa de detección de maliciosos (TDM) y la tasa de falsos positivos (TFP) se definen, como en el protocolo, sobre una muestra de URLs maliciosas verificadas y de URLs legítimas tomadas de

la lista Tranco [50]. La TDM cuenta las maliciosas que reciben el veredicto malicioso, con criterio de al menos el 90 %, y la TFP cuenta las legítimas que reciben ese mismo veredicto, con criterio de 5 % como máximo. Además se registra la tasa de advertencia, que suma los veredictos malicioso y sospechoso, porque un sospechoso sobre un sitio peligroso también protege al usuario.

7.5.3. Protocolo de verificación en dispositivo

El protocolo en el teléfono es una lista de verificación de aproximadamente cincuenta minutos, organizada en bloques con un resultado esperado por paso: arranque en frío y onboarding, análisis real contra el servidor, geocodificación inversa al aire libre, generación de la Story Card, camino sin conexión, y comportamiento del teclado y de las áreas seguras de la pantalla. La instrucción general es registrar qué falló y qué se observó, en lugar de marcar únicamente aprobado o reprobado.

Dos pasos no pueden omitirse. El primero consiste en intentar escanear con un segundo teléfono el código que aparece cubierto en la Story Card; si se lee, es un fallo grave de seguridad. El segundo verifica que la preferencia de ubicación sobreviva al reinicio, una regresión que se detectó y corrigió durante el desarrollo.

7.5.4. Métricas e instrumentos

El cuadro 7.4 asocia cada métrica del protocolo con el instrumento que la mide. Sin esa columna, una métrica puede enunciarse sin que exista un procedimiento capaz de producirla.

Tabla 7.4: Métricas del módulo y su instrumento de medición

Métrica	**Criterio**	**Instrumento**
Clasificación URL o no URL	al menos 98 %	Pruebas unitarias y banco de lectura
Cobertura de pruebas	al menos 70 %	Informe de cobertura
Persistencia del historial	100 %	Pruebas unitarias y protocolo en dispositivo
Reportes con carga completa	100 %	Pruebas unitarias
T_{scan} (percentil 90)	5 s o menos	Banco de lectura y prueba en dispositivo
Tasa de detección de códigos (TDR)	al menos 95 %	Banco de lectura y prueba en dispositivo
T_{F1} (percentil 90)	5 s o menos	Código real sobre muestras públicas
T_{total}	35 s o menos	Código real contra el servidor
Detección de maliciosos (TDM)	al menos 90 %	Muestra verificada; depende del servidor
Falsos positivos (TFP)	5 % o menos	Muestra de Tranco; depende del servidor
Geocodificación legible	al menos 85 %	Protocolo en dispositivo
Story Card generada	al menos 90 %	Protocolo en dispositivo
Comprensión del veredicto	al menos 90 %	Prueba de usabilidad
Finalización del flujo	al menos 85 %	Prueba de usabilidad
T_{learn}	60 s o menos	Prueba de usabilidad
T_{start}	3 s o menos	Medición en teléfono de gama media
Compatibilidad con Android 10 o superior	Sin errores críticos	Compilación y protocolo en dispositivo

7.6. Consideraciones éticas y de privacidad

La minimización de datos se trató como una decisión de método, porque determina qué puede medirse y cómo. La regla de mayor jerarquía del producto es que la aplicación nunca visita la URL analizada. Por lo tanto, ninguna medición del trabajo implica que el teléfono tenga contacto con la infraestructura del atacante ni revela que un usuario concreto está por acceder a una URL concreta.

De esa regla se derivan las demás. La geocodificación se resuelve en el dispositivo y las coordenadas no se envían al servidor del proyecto. Los reportes son anónimos y no incluyen identificador de usuario ni coordenadas. El historial reside solo en el teléfono. Las contraseñas de redes Wi-Fi codificadas en un QR se enmascaran en pantalla. Y el código que aparece en la Story Card se cubre con una capa opaca, porque compartir un código legible propagaría el ataque.

Para las pruebas de usabilidad se solicita el consentimiento informado de cada participante, se explica el propósito del estudio y el carácter voluntario de la participación, y los datos se tratan de forma anónima. Los participantes escanean únicamente códigos preparados para la prueba. Las URLs maliciosas de las muestras públicas se consultaron solo como texto contra los servicios de análisis, sin abrirlas en ningún navegador.

7.7. Limitaciones del método

La cobertura automatizada alcanza la lógica de negocio pero no las pantallas, cuyo comportamiento se verifica con protocolos manuales que dependen de quien los ejecuta. Las pruebas en hardware se realizan sobre un solo modelo de teléfono Android. La muestra de usabilidad es pequeña y sirve para encontrar problemas, no para estimar proporciones. Las mediciones contra el servidor corresponden a un ambiente de desarrollo en la capa gratuita de AWS, por lo que incluyen arranques en frío y límites que un despliegue de producción podría no presentar. Por último, las cuotas de los servicios gratuitos, en especial las de VirusTotal y PhishTank, limitan el tamaño de las muestras que pueden consultarse en un mismo día.

CAPÍTULO 8

Desarrollo del prototipo

Continuando con la metodología, este capítulo describe cómo se construyó el módulo móvil de QRisk y por qué se tomó cada decisión técnica. Empieza por la visión general del flujo y la arquitectura, sigue con cada etapa del recorrido del usuario y termina con las diferencias entre lo planificado en el protocolo y lo implementado.

8.1. Visión general del flujo

La figura 8.1 resume el recorrido de un código desde la cámara hasta el veredicto. La aplicación lee el código y clasifica su contenido. Si no es un enlace, muestra una pantalla informativa y no inicia ningún análisis. Si es un enlace, resuelve el filtro F1 en el propio teléfono y envía la URL junto con esa conclusión al servidor. El servidor corta la cascada si F1 ya concluyó; si no, ejecuta los filtros heurístico y de comportamiento. Con la respuesta, la aplicación determina el veredicto, lo guarda en el historial y lo presenta al usuario.

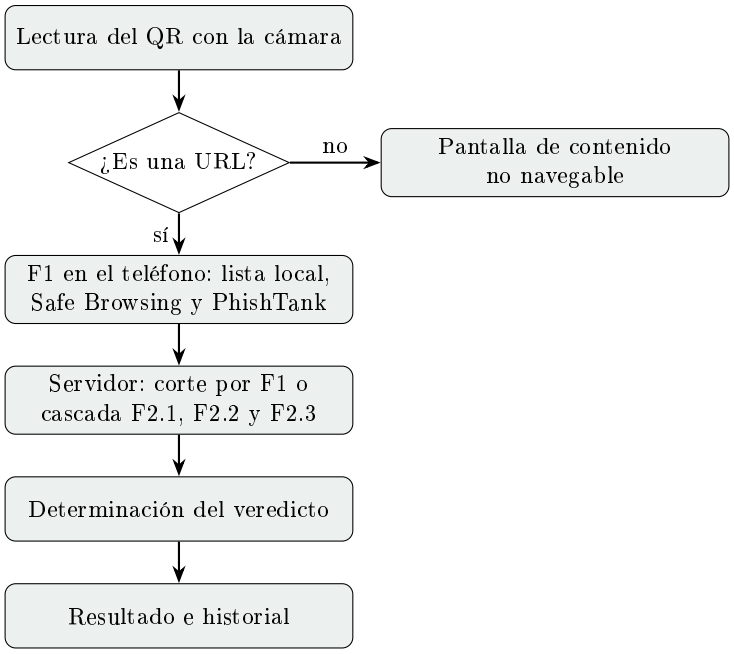


Figura 8.1: Recorrido de un código QR dentro de QRisk

8.2. Plataforma y versiones

El prototipo se construyó sobre las versiones del cuadro 8.1. Se registran las versiones exactas porque varias decisiones se explican por restricciones de compatibilidad entre versiones concretas.

Tabla 8.1: Versiones del entorno de desarrollo

Componente	**Versión**
Expo SDK	57.0.12
React Native	0.86.2, con la New Architecture
React	19.2.3
Enrutamiento	Expo Router 57
Cámara y lectura de códigos	`expo-camera` 57
Almacenamiento clave valor	`expo-sqlite` 57
Ubicación	`expo-location` 57
Lenguaje	TypeScript 6.0.3 en modo estricto
Pruebas	Jest 30 con el preset `jest-expo`
Compilación	EAS Build; Node 20.20.2, JDK 17, Gradle 9.3.1
Android mínimo	API 24

Durante la configuración apareció un conflicto entre dependencias nativas del propio SDK. El enrutador arrastraba la versión 0.12 de la librería de *worklets*, mientras que el núcleo de módulos de Expo solo admitía hasta la 0.10, y la combinación rompía la compilación nativa de Android con un error de símbolo ausente. Se resolvió fijando explícitamente un par de versiones compatibles en el manifiesto de dependencias. Además, la compilación local con Gradle se detuvo repetidamente en el equipo de desarrollo sin producir el instalable, mientras que el paquete de JavaScript se exportaba

sin errores; por eso la compilación se trasladó a EAS Build, que compila en la nube.

8.3. Arquitectura de la aplicación

La aplicación se organiza en cuatro capas con responsabilidades que no se solapan, según el cuadro 8.2. Esta separación es la que permite probar la lógica de negocio sin cámara, sin red y sin dispositivo.

Tabla 8.2: Capas de la aplicación

Capa	**Responsabilidad**
Rutas	Pantallas con enrutamiento por archivos: dos pasos de onboarding, escáner, carga, resultado, detalle, contenido no navegable, historial, reporte y Story Card, más una ruta inicial que decide a dónde dirigir al usuario
Sistema de diseño	Diecisiete componentes propios, sin librerías de interfaz externas, y los tokens de color, tipografía y espaciado del prototipo visual
Servicios	Lógica de negocio pura: análisis, cliente HTTP y errores, clasificación de QR, almacenamiento, ubicación, reporte y formato
Estado	Un único contexto de React con el resultado vigente, el historial y la preferencia de ubicación

Las pantallas nunca ven la respuesta cruda del servidor. Consumen un resultado de análisis con forma propia de la aplicación, y la traducción ocurre en un único adaptador. Gracias a eso, los cambios en la respuesta del servidor se absorbieron en un solo archivo durante toda la integración.

8.4. Escaneo y clasificación del contenido

El escáner usa la vista de cámara de `expo-camera` restringida al formato QR, con control de linterna y gestión de permisos. Después de la primera lectura válida el escáner se bloquea, para evitar que un mismo código dispare varios análisis mientras sigue frente a la cámara. Los permisos de cámara y de ubicación se solicitan en el onboarding, con una explicación previa de para qué se usan, y no encima del escáner, donde el diálogo del sistema interrumpiría la primera lectura.

La clasificación separa el contenido en tres tipos: enlace, red Wi-Fi y otro contenido. Se consideran enlaces las URLs con esquema `http` o `https` y los dominios escritos sin esquema, a los que se antepone `https`. Los demás contenidos (contactos, correos, teléfonos, coordenadas y texto) se muestran en una pantalla informativa con su etiqueta, y las contraseñas de Wi-Fi se enmascaran. Ningún contenido no navegable se envía al servidor.

8.5. El filtro F1 en el dispositivo

El contrato de interfaz espera que el cliente resuelva el filtro F1 y envíe su conclusión al servidor. Si F1 concluye seguro o malicioso, el servidor corta la cascada y responde en pocos segundos, en lugar de ejecutar todos los filtros. Por esa razón F1 se implementó en la aplicación, con tres fuentes.

La primera es una lista local de seis dominios de reputación indiscutida, que solo puede concluir seguro. Dos reglas impiden que esa lista se convierta en una vía de evasión. La coincidencia se hace por dominio completo o subdominio separado por punto, nunca por fragmentos de texto: `google.com.phishing.top` contiene la cadena "google.com" y `no-github.com` termina en "github.com", y ninguno de los dos debe pasar. Además, solo se aceptan URLs con HTTPS, porque un dominio legítimo servido sin cifrado sigue siendo interceptable. No existe una lista negra local, a propósito, porque quedaría desactualizada el mismo día de la compilación.

La segunda y la tercera fuentes son Google Safe Browsing y PhishTank, que solo pueden concluir malicioso. Se consultan en paralelo, cada una con un tiempo límite de 4 s, de modo que T_{F1} queda acotado por la consulta más lenta y no por la suma de ambas. Safe Browsing se consulta sobre las categorías de malware e ingeniería social. Si alguna fuente no responde, el filtro sigue con las respuestas disponibles y la pantalla de detalle indica que esa fuente no pudo consultarse, en lugar de afirmar que la URL no estaba reportada.

El orden de evaluación es la decisión más importante del filtro. La intuición diría que, si la lista local aprueba el dominio, se puede cortar ahí y ahorrar la consulta de red. Eso es exactamente lo que no se hace. Un dominio confiable puede alojar una página maliciosa concreta, como un repositorio con un ejecutable, y Safe Browsing opera sobre la URL completa, no solo sobre el dominio. Si la lista local cortara, a un atacante le bastaría alojar su página bajo uno de esos dominios. Por lo tanto, un seguro de la lista local es una hipótesis que cualquiera de las fuentes remotas puede vetar, y un malicioso siempre tiene prioridad.

El protocolo incluía VirusTotal dentro de F1. En la implementación final, VirusTotal se consulta dentro del filtro F2.2 del servidor, por una razón de cuotas: su plan gratuito permite 4 consultas por minuto y 500 por día, y consultarlo también desde cada teléfono gastaría dos veces la cuota más estrecha del sistema. De esta forma, las tres fuentes del protocolo siguen participando, cada una en la capa donde es técnicamente viable.

8.6. Integración con la cascada del servidor

El análisis se solicita con una sola llamada que envía la URL y la conclusión de F1. Dentro del servidor, la cascada sigue este orden: si F1 concluyó, ese es el veredicto final; si no, se consulta la caché de análisis recientes; después se ejecuta F2.1, que calcula un puntaje heurístico de 0 a 100 y corta con sospechoso si supera 60; y por último F2.2, que consulta VirusTotal y URLScan.io y, cuando puede, obtiene la captura del sitio que corresponde a F2.3. La respuesta indica qué filtro decidió.

La captura de F2.3 no se trató como una etapa independiente, porque es un subproducto de F2.2: si la cascada corta antes, no hay captura. Cuando existe, se muestra como imagen estática y nunca dentro de un navegador embebido, porque una página cargada en la aplicación podría ejecutar código o hacerse pasar por una pantalla de QRisk. La dirección de la imagen caduca en una hora, por lo que no se guarda en el historial.

Al verificar el adaptador contra respuestas reales apareció un error de presentación: cuando F1 concluía seguro, la vista de detalle mostraba los filtros posteriores como "sin completar", lo que sugería que algo había fallado. No era cierto: la cascada no falló, concluyó antes, que es su objetivo. La vista se corrigió para explicar qué filtro decidió y por qué los demás no fueron necesarios.

8.7. Determinación del veredicto

El veredicto se resuelve con tres fuentes en orden de precedencia. La primera es el servidor: si concluyó seguro, sospechoso o malicioso, su palabra se respeta. La segunda es el puntaje heurístico, que solo se usa cuando el servidor responde "pendiente". La tercera es la evidencia de comportamiento de F2.2, que solo puede elevar un sospechoso a malicioso.

La derivación por puntaje existe porque el servidor deja un tramo sin cubrir. Su umbral de F2.1 es estrictamente mayor que 60, por lo que una URL con puntaje de 55, que ya reúne varias señales simultáneas, regresa como pendiente. Sin la derivación, esa URL se vería como incierta con la evidencia a la vista. Los cortes, recogidos en el cuadro 8.3, se alinearon con los pesos que el propio servidor asigna a cada criterio: dominio de nivel superior de riesgo, 25 puntos; host que es una dirección IP, 30; ausencia de HTTPS, 15; y palabra clave sospechosa, 15.

Tabla 8.3: Bandas de puntaje y su fundamento

Puntaje	**Veredicto**	**Fundamento**
0 a 29	Seguro	Un solo criterio menor no basta. Marcar como riesgoso todo sitio sin HTTPS produciría tantos falsos positivos que la aplicación perdería credibilidad
30 a 59	Sospechoso	30 es el peso de un host que es una IP y también la suma de dos criterios menores. Merece una advertencia, no una condena
60 a 100	Malicioso	Tres o más señales simultáneas, como IP, ausencia de HTTPS y la palabra *login*: el caso clásico de phishing

La elevación por evidencia responde a otra limitación del servidor: por comportamiento, su veredicto máximo es sospechoso, de modo que un sitio que 18 motores de VirusTotal marcan como malicioso recibe la misma etiqueta que un acortador marcado por dos. La aplicación eleva a malicioso cuando al menos tres motores coinciden, o cuando URLScan concluyó que el sitio es malicioso o eliminó su informe tras analizarlo. Se eligieron tres motores porque uno solo puede ser un falso positivo de un producto agresivo y dos pueden compartir firmas, mientras que tres fuentes independientes coincidiendo ya es evidencia difícil de discutir.

Ninguna de las dos derivaciones inventa un veredicto. El puntaje, los criterios y el recuento de motores provienen del servidor y de los servicios que consulta; la aplicación solo aporta los cortes. Además, ambas se desmontan solas: el día que el servidor concluya por su cuenta, su veredicto tendrá precedencia y estas reglas dejarán de cambiar el resultado sin modificar el código.

El puntaje se valida antes de usarse. Un valor ausente, no numérico o fuera del rango de 0 a 100 no produce ningún veredicto y el resultado se resuelve como incierto. La razón es que un -20 o un 150 no significan "muy seguro" o "muy malicioso", sino que el servidor cambió de escala y los cortes ya no significan lo mismo. Antes de esta validación, un puntaje negativo producía un seguro con apariencia de correcto, que es el peor error posible del producto.

8.8. Política de tiempos y degradación

El tiempo límite del análisis cambió varias veces durante el proyecto, siempre a partir de mediciones contra el servidor real. El cuadro 8.4 resume esa historia.

Tabla 8.4: Evolución de la latencia del servidor y del tiempo límite

Fecha	Latencia observada	Decisión
20/09/2026	5,5 s la primera llamada por arranque en frío; de 1,6 a 3,1 s las siguientes	Límite inicial
30/09/2026	De 14,2 a 14,5 s para una URL nueva, por la captura de F2.3	Límite de 25 s y aviso de espera a los 8 s
11/10/2026	Corte por F1 en 5,3 y 5,5 s; cascada completa entre 24 y 35 s; algunas respuestas HTTP 504 a los 35 s	Límite de 40 s y aviso a los 20 s
12/10/2026	Casi toda URL nueva llega al corte de 35 s; un segundo intento puede completar en 26,7 a 30,9 s	Presupuesto de 90 s con un reintento

El límite que importa no lo pone la aplicación. API Gateway corta cada petición a los 35 s y responde HTTP 504 mientras la función sigue trabajando, y la cascada completa tarda alrededor de ese mismo tiempo. Por eso, desde el 12 de octubre, la aplicación tiene un presupuesto total de 90 s con un único reintento tras un 504, que solo se lanza si quedan al menos 30 s para que pueda completarse. El reintento funciona porque los servicios externos ya analizaron la URL en el primer intento y responden más rápido; aun así, no resuelve el problema de fondo, que corresponde al servidor.

Es importante distinguir el tiempo límite de la métrica. T_{total} mide cuánto tarda el servidor en responder; el tiempo límite define cuándo se rinde la aplicación. Recortar el límite no mejora el número medido, solo convierte análisis buenos en inciertos.

La pantalla de carga muestra las etapas de la cascada con duraciones proporcionales a lo medido (3, 8 y 16 s) mediante una animación de esqueleto, y avisa a los 20 s que la espera se está prolongando. El aviso pasó de 8 a 20 s porque con la cascada completa aparecía en casi todos los análisis, y un aviso que aparece siempre deja de informar.

El cliente HTTP traduce cualquier fallo a un vocabulario cerrado de once códigos, para que la interfaz decida según el código y nunca interpretando el texto del mensaje. Los códigos se agrupan en reintentables (tiempo agotado, filtro no disponible, límite de consultas, error interno y fallo de red), no reintentables (URL inválida, carga inválida y análisis no encontrado) y la cancelación, que se distingue del tiempo agotado porque salir de la pantalla no es un error.

La política de degradación tiene una regla central: ningún fallo produce un veredicto seguro. Cualquier error degrada a un resultado incierto con un aviso visible de que no se completaron todas las verificaciones. Un incierto honesto es preferible a un seguro calculado con información incompleta.

8.9. Pantallas de resultado

El resultado simple muestra un emblema, el veredicto en palabras, el dominio y un resumen breve. Cada veredicto tiene un color y un texto propios, como se resume en el cuadro 8.5. La jerarquía de los botones se invierte según el riesgo: en un sitio seguro, la acción principal es abrir el sitio; cuando hay señales de riesgo, la acción principal es ver los detalles.

Tabla 8.5: Veredictos y su significado para el usuario

Veredicto	**Mensaje al usuario**
Seguro	Sin señales de riesgo
Sospechoso	Hay señales de riesgo; abrirlo solo si se confía en quien colocó el código
Malicioso	Peligroso; no abrirlo ni ingresar datos
Incierto	No se pudo completar el análisis; tratarlo con precaución

El resultado extendido aplica el principio de revelación progresiva: la advertencia principal debe entenderla cualquier persona, y el detalle técnico queda disponible para quien lo busque. Incluye un medidor de riesgo con el puntaje heurístico y una sección por filtro con los criterios reales que devolvió el servidor, como "Host es una IP: 192.168.1.1" o "18 motores lo marcan como malicioso", y no un texto genérico. Cuando el puntaje no existe porque la cascada cortó antes, el medidor no se dibuja vacío, sino que se explica qué filtro decidió.

8.10. Abrir el sitio analizado

En una primera versión, el botón de abrir mostraba un mensaje que decía que se abría el sitio, pero no abría nada. Se evaluaron tres alternativas: no abrir nunca, copiar la URL al portapapeles o abrirla con fricción proporcional al riesgo. No abrir nunca era coherente con la regla de seguridad, pero el usuario lo haría de todos modos copiando la URL a mano, sin ninguna advertencia de por medio. Copiar al portapapeles dejaba a la persona a un solo paso del sitio, con la misma exposición y menos información.

Se eligió la tercera opción. En un sitio seguro, el enlace se abre directamente en el navegador del sistema. Con cualquier otro veredicto, incluido el incierto, aparece una hoja de confirmación que repite el dominio y cuántas señales de riesgo se encontraron, con "Cancelar" como botón principal. El sitio siempre se abre fuera de QRisk, en el navegador del sistema, donde la barra de direcciones es una defensa adicional. La aplicación informa y encarece la decisión riesgosa, pero no la prohíbe, porque quien sabe si confía en quien colocó el código es la persona.

8.11. Historial local

El historial guarda los últimos 20 escaneos y descarta el más antiguo al superar ese límite. Cada entrada conserva el análisis completo, de modo que al abrirla se recupera su veredicto sin conexión. Se almacena en el almacén clave valor de `expo-sqlite`, cuya lectura síncrona permite cargar el historial en el primer renderizado, sin que la lista aparezca vacía por un instante.

El mismo almacén guarda si el usuario ya completó el onboarding y si concedió la ubicación. Al hacer que el onboarding se mostrara una sola vez, la preferencia de ubicación quedó viviendo solo en memoria y, tras el primer reinicio, el historial dejaba de registrar el lugar sin que nadie lo notara. La corrección fue persistir ambas preferencias juntas. Las tres lecturas fallan hacia el lado seguro: si el almacén no responde, el historial aparece vacío, el onboarding se repite y la ubicación queda desactivada, pero la aplicación no se cae.

8.12. Reporte de códigos maliciosos

El formulario de reporte permite enviar al servidor la URL, un motivo opcional y el identificador del análisis cuando existe. El envío es anónimo: no incluye identificador de usuario, coordenadas ni la hora local, y los campos opcionales solo se envían si tienen contenido. Si el envío falla, el texto escrito no se pierde. Durante las pruebas de integración, el servicio de reportes validaba correctamente las entradas, pero respondía con error interno al guardar; la aplicación distingue ese caso de un fallo de red, para no pedirle al usuario que revise una conexión que está bien.

8.13. Story Card

La Story Card es una imagen que el usuario puede compartir para advertir a otras personas sobre un código. Reúne cuatro elementos: una foto del entorno donde está el código, el código cubierto, el nombre del lugar y el veredicto. La foto se toma con la cámara o se elige de la galería, y la pantalla explica para qué sirve: mostrar dónde está el código para que otras personas lo reconozcan. La imagen se genera capturando la vista de la tarjeta y puede compartirse o guardarse en la galería.

El protocolo hablaba de un QR difuminado. En la implementación el código se cubre con una capa opaca, porque, como se explicó en el marco teórico, la corrección de errores del estándar permite leer códigos parcialmente dañados, y un código borroso puede seguir siendo legible. Compartir una tarjeta con un código legible propagaría justamente el ataque que se quiere advertir.

8.14. Geocodificación inversa

El nombre del lugar se obtiene con `expo-location`, que resuelve la geocodificación con el servicio del sistema operativo. Las coordenadas no se envían al servidor del proyecto y en el historial se guarda solo el nombre legible del lugar. La ubicación se resuelve en paralelo con el análisis, para que nunca retrase el veredicto. Si falla o el usuario no concedió el permiso, el resultado es nulo y la interfaz simplemente oculta esa fila.

8.15. Diferencias entre el protocolo y lo implementado

El cuadro 8.6 resume las diferencias entre lo planificado en el protocolo y lo construido. Cada una responde a una razón técnica documentada a lo largo de este capítulo.

Tabla 8.6: Diferencias con el protocolo y su justificación

Protocolo	**Implementación**	**Justificación**
`react-native-vision-camera`	`expo-camera`	Módulo alineado con las versiones del SDK; reduce conflictos nativos como el observado con los *worklets*. La medición de lectura decide si se mantiene
`react-native-mmkv` con cifrado	Almacén de `expo-sqlite`, sin cifrado	Ofrece la lectura síncrona que necesita el primer renderizado. El historial no contiene credenciales ni coordenadas; el cifrado se deja como recomendación
F1 con Safe Browsing, PhishTank y VirusTotal	Safe Browsing y PhishTank en la app; VirusTotal en F2.2	Evita gastar dos veces la cuota más estrecha del sistema
Tiempo de respuesta de F1 menor a 5 s	Tope de 4 s por fuente en paralelo	El tope de F1 cumple el criterio; la cascada completa tiene su propio criterio
QR difuminado	Capa opaca	Un código borroso puede seguir siendo legible
React Navigation	Expo Router	Enrutamiento por archivos del mismo ecosistema
iOS y Android	Solo Android	No se contó con hardware de Apple; la configuración de iOS se conserva

CAPÍTULO 9

Resultados

Este capítulo presenta los resultados obtenidos al evaluar el prototipo según el diseño descrito en la metodología. Se sigue una regla única: cada número viene de una corrida real con fecha, y lo que todavía no se ha medido se declara como pendiente. La evidencia de cada medición se encuentra en el anexo A.

Los resultados se ordenan de lo que no depende del hardware ni del servidor a lo que sí depende de ellos: primero la verificación de la lógica de negocio, después la compilación, el filtro F1, la cascada completa, la lectura de códigos, la verificación en el teléfono y la usabilidad. El capítulo cierra con un cuadro de cumplimiento de las diecisiete métricas y con las limitaciones de los resultados.

9.1. Verificación de la lógica de negocio

La corrida de referencia de esta sección se realizó el 11 de octubre de 2026, con Node 20.20.2 sobre Windows 11, Expo SDK 57.0.12 y React Native 0.86.2. En esa fecha, el conjunto completo sumaba 249 pruebas en 19 suites, todas aprobadas, y la verificación de tipos de TypeScript terminaba sin errores. El cuadro 9.1 resume las cuatro métricas que se cubren en este nivel.

Tabla 9.1: Métricas verificadas con pruebas automatizadas (11 de octubre de 2026)

Métrica	**Criterio**	**Obtenido**	**Cumple**
Cobertura de pruebas	al menos 70 %	99,72 % de sentencias y 100 % de líneas	Sí
Clasificación URL o no URL	al menos 98 %	100 % (18 de 18 casos)	Sí
Persistencia del historial	100 %	100 % (36 de 36 casos)	Sí
Reportes con carga completa	100 %	100 % (16 de 16 casos)	Sí

La cobertura se distribuyó de forma pareja entre los módulos de servicios: análisis, cliente HTTP,

formato, ubicación, clasificación de QR, reporte y almacenamiento alcanzaron el 100 % de líneas y de funciones. La pequeña diferencia en sentencias corresponde a dos guardas defensivas del módulo de errores y a archivos que solo contienen declaraciones de tipos.

Los 18 casos de clasificación cubren URLs con esquema, dominios sin esquema, credenciales Wi-Fi, contactos, correos, teléfonos, coordenadas y texto suelto. Los casos de persistencia verifican guardar, leer, el límite de 20 entradas, el descarte del registro más antiguo y la recuperación ante datos corruptos. Los casos de reporte verifican que la carga se arme completa y anónima. Es importante precisar el alcance de esta última métrica: se valida la construcción de la carga, no su recepción en el servidor, cuyo servicio de guardado respondía con error interno en las pruebas de integración.

El conjunto de pruebas creció con la integración de la cascada y del filtro F1. Tras la integración del 11 de octubre sumaba 307 pruebas en 20 suites, con 99,75 % de cobertura de sentencias y 100 % de líneas y funciones. Tras implementar Safe Browsing, ese mismo día, sumaba 338 pruebas en 22 suites, con 100 % de cobertura en los tres archivos de F1.

Pendiente: Volver a correr `npm run test:coverage` sobre la versión final del código y actualizar las cifras de esta sección y del cuadro 9.1 con la fecha de esa corrida. Después del 11 de octubre se agregaron pruebas de PhishTank, de la elevación por evidencia y de las métricas de detección.

9.2. Compilación reproducible

El instalable para Android se generó con EAS Build. En el historial del proyecto había ocho compilaciones del perfil de prueba, de las cuales siete terminaron correctamente y una fue cancelada. La compilación de referencia, del 11 de octubre de 2026, tardó 13 min y 25 s y corresponde exactamente al mismo commit sobre el que se midieron las pruebas automatizadas. Esto permite afirmar que las mediciones en hardware se realizan sobre la misma versión del código que la verificación automatizada. Además, que siete de ocho compilaciones terminaran en un entorno limpio indica que los bloqueos de Gradle observados en el equipo de desarrollo eran propios de ese equipo y no del proyecto.

El teléfono de pruebas es un Samsung Galaxy A11 (SM-A115M) con Android 11 (API 30), procesador Qualcomm de ocho núcleos, cerca de 1,8 GB de memoria RAM, pantalla de 720 por 1560 píxeles y cámara con enfoque automático. Por su memoria y su procesador corresponde a la gama de entrada, por debajo de la gama media sobre la que el protocolo define el criterio de T_{start}. Los tiempos medidos en este equipo deben leerse, por lo tanto, como una estimación conservadora.

9.3. Filtro F1 de reputación

9.3.1. Verificación funcional en vivo

El 11 de octubre de 2026 se ejecutó la lógica del filtro F1 contra Google Safe Browsing en vivo, con ocho casos de control. Los ocho produjeron el resultado esperado, con tiempos de entre 72 y 324 ms. Las dos URLs de prueba oficiales de Google, de malware y de phishing, se clasificaron como maliciosas; los dominios de la lista local, como seguros; y tres casos de evasión dieron incierto, como se esperaba: un dominio desconocido, un dominio confiable servido sin HTTPS y un dominio que imita a uno confiable.

9.3.2. F1 sobre la muestra de Kaggle

El 12 de octubre de 2026 se ejecutó el código real de la aplicación sobre 2 000 URLs del conjunto público de Kaggle [21], con 500 por clase y semilla fija. Ninguna consulta superó el tiempo límite. El cuadro 9.2 resume los tiempos.

Tabla 9.2: Tiempo del filtro F1 sobre muestras de Kaggle (12 de octubre de 2026)

Configuración	URLs	Mediana	Perc. 90	Máximo
Lista local y Safe Browsing	2 000	137 ms	214 ms	699 ms
Lista local, Safe Browsing y PhishTank	400	406 ms	488 ms	1 106 ms

En ambas configuraciones el percentil 90 de T_{F1} quedó muy por debajo del criterio de 5 s. Estas corridas se hicieron desde una computadora con red fija y no desde el teléfono con red móvil, como pide el protocolo, de modo que los tiempos en el dispositivo serán mayores.

Pendiente: Medir T_{F1} desde el teléfono de pruebas con red móvil 4G, como especifica el protocolo, y agregar la fila correspondiente al cuadro 9.2.

La incorporación de PhishTank multiplicó la detección de F1 en la clase de phishing. Sobre 100 URLs de esa clase, F1 marcó como maliciosas 2 sin PhishTank y 14 con PhishTank. Safe Browsing y PhishTank no coincidieron en ninguna URL, lo que indica que cubren casos distintos. PhishTank no produjo falsos positivos en las 70 URLs benignas que respondió, y no detectó ninguna de las clases de malware ni de desfiguración, lo que es consistente con que se trata de una base especializada en phishing.

La medición mostró también la fragilidad operativa de PhishTank. Sin una clave de aplicación, su servicio cortaba las consultas con HTTP 429 después de unas 66 consultas seguidas, un límite que no aparece en su documentación. Aun espaciando las consultas, 104 de 400 (26 %) quedaron sin respuesta. En la aplicación, cada uno de esos fallos se mostró como una fuente no consultada y el análisis continuó, lo que verificó la degradación con fallos reales y no simulados.

La muestra reveló además un hallazgo sobre la lista local. Cuatro URLs de la clase de malware recibieron seguro porque eran descargas alojadas en servicios con contenido subido por usuarios, como Google Drive y archivos de repositorios de GitLab. La lista acepta los subdominios de sus dominios confiables, y esos servicios viven bajo dominios de la lista. Este caso se retoma en las recomendaciones.

9.3.3. Validez de la muestra de Kaggle

Al revisar a mano las URLs que contaron como error se encontró que el conjunto de Kaggle no es adecuado para medir la detección. Varias URLs etiquetadas como benignas eran phishing real mal etiquetado: Safe Browsing o varios motores de VirusTotal las marcaban como maliciosas. Varias etiquetadas como phishing eran sitios legítimos. Los sitios de la clase de desfiguración eran páginas atacadas hacia 2016 que probablemente ya fueron restauradas. Por otro lado, el formato de la URL delataba la etiqueta: el 92 % de las benignas no tenía esquema y el 100 % de las de desfiguración sí.

Por lo tanto, la muestra de Kaggle se utilizó solo para medir propiedades que no dependen de la etiqueta, como T_{F1} y la tasa de cortes del servidor, y no para la TDM ni la TFP. Esas dos métricas requieren una muestra fresca y etiquetada en la fecha de la prueba.

9.4. Cascada completa de filtros

9.4.1. Tiempo total y cortes del servidor

El 12 de octubre de 2026 se ejecutó la cascada completa sobre 100 URLs de la muestra de Kaggle, 50 maliciosas y 50 legítimas, consultadas en serie. El hallazgo principal fue operativo: 45 de las 99 consultas con URL válida terminaron en HTTP 504 al llegar al límite de 35 s de API Gateway, y la aplicación las mostró como inciertas. Esta es la causa principal de la alta proporción de resultados inciertos, y no depende de la aplicación.

En las 54 respuestas exitosas, T_{total} tuvo una mediana de 26,8 s, un percentil 90 de 31,3 s y un máximo de 34,9 s. El criterio de 35 s se cumple, pero solo porque lo que supera ese tiempo ya no llega a la aplicación. En esas mismas respuestas, el servidor decidió por F2.2 con sospechoso en 18 casos, por F1 con seguro en 3 y con malicioso en 1, y devolvió pendiente en 32.

La política de tiempos de la aplicación recuperó parte de esos cortes. El 12 de octubre se observó que, tras un 504, una segunda consulta de la misma URL podía completar en 26,7 a 30,9 s, porque los servicios externos ya la habían analizado. El reintento implementado rescató una de cada tres URLs nuevas en esa prueba.

Espacio reservado para la figura
Distribución de T_{total} en las respuestas exitosas y proporción de cortes por HTTP 504

Figura 9.1: Distribución de T_{total} en las respuestas exitosas y proporción de cortes por HTTP 504

9.4.2. Cobertura de cada filtro sobre amenazas activas

El 12 de octubre de 2026 se consultaron contra F1 22 URLs maliciosas activas del registro reciente de URLhaus [17], varias reportadas pocas horas antes. Safe Browsing no detectó ninguna, ni ampliando la consulta a los cuatro tipos de amenaza que ofrece la API. Para descartar un error de integración se consultaron en la misma sesión dos URLs de control de Google, y ambas fueron detectadas. El resultado, entonces, describe la cobertura de Safe Browsing y no un fallo de la consulta.

La explicación está en el tipo de amenaza. Las 22 URLs eran infraestructura de distribución de malware: direcciones IP con puertos altos que sirven scripts, o archivos maliciosos subidos a dominios legítimos comprometidos. Safe Browsing está optimizado para proteger a una persona que navega, y estas URLs no se navegan, sino que las descarga software malicioso ya instalado.

Al pasar algunas de estas URLs por el resto de la cascada se observó que cada filtro cubre algo que los otros no ven, según el cuadro 9.3.

Tabla 9.3: Qué filtro detecta cada tipo de amenaza (12 de octubre de 2026)

Tipo de URL	F1	F2.1	F2.2
IP literal con puerto alto que sirve un script	No	IP literal, 30 puntos	Corte por 504
Script en un dominio legítimo comprometido	No	Puntaje 0	Corte por 504
Imagen con carga maliciosa en un dominio de aspecto normal	No	Puntaje 0	Sospechoso

El tercer caso es el más ilustrativo. Su puntaje heurístico es cero y Safe Browsing no lo conoce, pero VirusTotal sí. Con un solo filtro, esa URL habría pasado como benigna. Además, ninguna de las tres URLs fue detectada por más de una vía. Este resultado es el argumento empírico de por qué el sistema se diseñó como una cascada y no como un filtro único.

9.4.3. Elevación por evidencia de comportamiento

El 12 de octubre se aplicó la regla de elevación sobre respuestas reales del servidor. Una imagen con carga maliciosa que 14 motores de VirusTotal marcaban como maliciosa, y que el servidor devolvía como sospechosa, se elevó a malicioso. Un acortador marcado por dos motores y otro marcado por uno se mantuvieron como sospechosos, porque no alcanzan el umbral de tres motores. Una URL de phishing con puntaje heurístico de 95 también se mantuvo como sospechosa, porque la regla solo considera la evidencia de comportamiento.

9.4.4. Detección sobre la muestra de Kaggle

Como referencia, y con las limitaciones de etiquetado ya descritas, el cuadro 9.4 muestra las métricas de detección calculadas sobre las 100 URLs de la cascada, sin corregir las etiquetas del conjunto.

Tabla 9.4: Métricas de detección sobre la muestra de Kaggle, sin corregir etiquetas

Métrica	Valor	Criterio
Maliciosas con veredicto malicioso, sin PhishTank	26 % (13 de 50)	al menos 90 %
Maliciosas con veredicto malicioso, con PhishTank	28 % (14 de 50)	al menos 90 %
Maliciosas con malicioso o sospechoso	36 %	no definido
Legítimas con veredicto malicioso	6 % (3 de 50)	5 % o menos

Estas cifras no miden la capacidad de detección del sistema. Las tres URLs “legítimas” que recibieron malicioso eran phishing real mal etiquetado, lo que lleva la tasa real de falsos positivos a cero en esta muestra. Por el lado de las maliciosas, gran parte de los inciertos se debe a los cortes por 504 y no a una falta de detección. Se reportan para dejar constancia de la corrida, no como resultado de las métricas del protocolo.

9.5. Detección con una muestra verificada

Pendiente: Ejecutar la medición de TDM y TFP con una muestra fresca, etiquetada en la fecha de la prueba: 50 URLs de phishing verificadas recientemente (PhishTank u OpenPhish) y 50 URLs legítimas de la lista Tranco. Registrar la fuente y la fecha de extracción, y reportar cuatro cifras: TDM estricta, tasa de advertencia (malicioso más sospechoso), TFP estricta y legítimas con sospechoso. Indicar cuántas consultas terminaron en 504.

Tabla 9.5: Detección sobre la muestra verificada (por completar)

Métrica	Obtenido	Criterio
TDM estricta (malicioso)		al menos 90 %
Tasa de advertencia (malicioso o sospechoso)		no definido
TFP estricta (legítimas con malicioso)		5 % o menos
Legítimas con sospechoso		no definido
Consultas cortadas por HTTP 504		no definido

9.6. Lectura de códigos QR

Esta sección presenta los resultados del banco de pruebas automatizado de lectura descrito en la metodología, que mide T_{scan}, la tasa de detección de códigos y la precisión de clasificación sobre un conjunto de imágenes con distintas condiciones de tamaño, ángulo, iluminación y contenido.

Pendiente: Completar con los resultados del banco de pruebas automatizado: cantidad de imágenes procesadas, equipo donde corrió, resultados globales, resultados por condición y gráficas. Discutir si el fallo, en caso de haberlo, se concentra en un solo eje (ángulo, iluminación o tamaño) y concluir la decisión sobre la librería de cámara.

Tabla 9.6: Resultados globales de lectura (por completar)

Indicador	Obtenido	Criterio
Imágenes procesadas		
Tasa de detección (TDR)		al menos 95 %
T_{scan}, mediana		
T_{scan}, percentil 90		5 s o menos
T_{scan}, máximo		
Precisión de clasificación		al menos 98 %

Tabla 9.7: Resultados de lectura por condición (por completar)

Condición	**Nivel**	**n**	**TDR**	T_{scan} **p90**
Tamaño	Pequeño			
	Mediano			
	Grande			
Ángulo	De frente			
	30°			
	45°			
Iluminación	Luz de día			
	Penumbra			
	Oscuridad con linterna			
Contenido	URL			
	No URL			

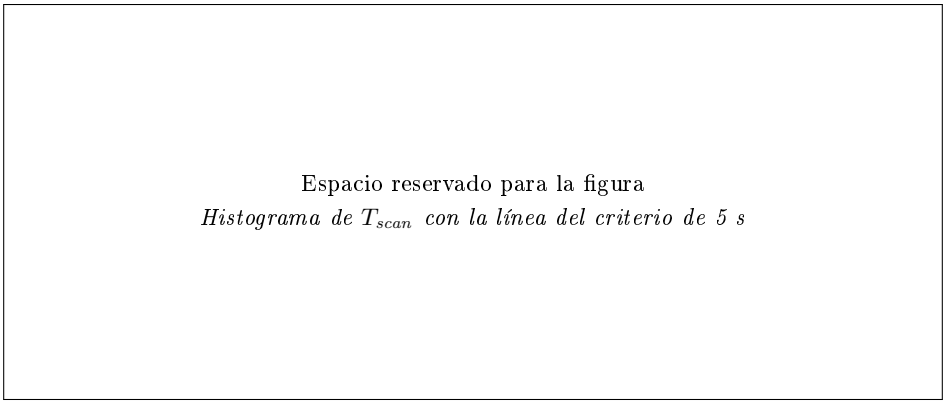


Figura 9.2: Histograma de T_{scan} con la línea del criterio de 5 s

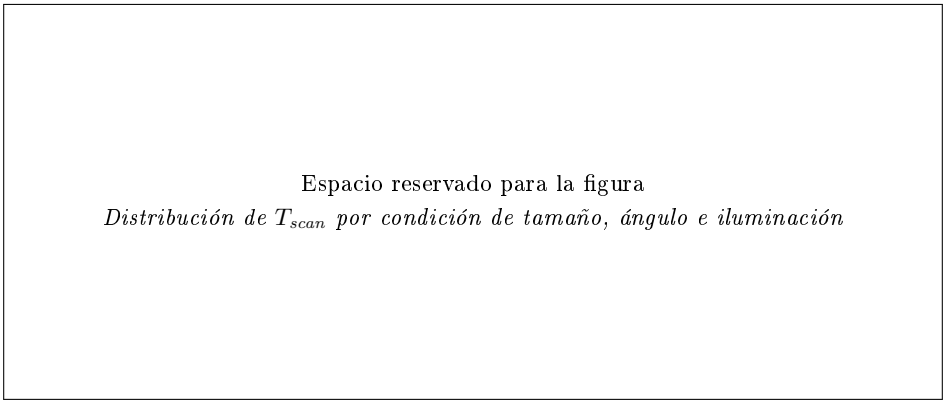


Figura 9.3: Distribución de T_{scan} por condición de tamaño, ángulo e iluminación

Pendiente: Decisión sobre la librería de cámara: indicar si se mantiene `expo-camera` o si corresponde migrar a `react-native-vision-camera`, según el criterio de la sección 7.5.

9.7. Verificación en el dispositivo

Pendiente: Completar con los resultados del protocolo en el teléfono: onboarding una sola vez, análisis real con los tres veredictos, geocodificación en 10 ubicaciones, Story Card en 10 escaneos, prueba de legibilidad del QR cubierto con un segundo teléfono, preferencia de ubicación tras el reinicio, camino sin conexión, T_{start} en cinco arranques en frío y compatibilidad con Android 11. Agregar capturas de pantalla de cada veredicto.

Tabla 9.8: Resultados del protocolo en dispositivo (por completar)

Verificación	**Obtenido**	**Criterio**
Geocodificación legible (10 ubicaciones)		al menos 85 %
Story Card con los cuatro elementos (10 escaneos)		al menos 90 %
QR cubierto ilegible desde otro teléfono		Ilegible
Preferencia de ubicación tras reinicio		Se conserva
Camino sin conexión		Incierto, sin fallo
T_{start}, mediana de 5 arranques		3 s o menos
Errores críticos en Android 11		Ninguno

Espacio reservado para la figura

Pantallas de resultado para los veredictos seguro, sospechoso y malicioso

Figura 9.4: Pantallas de resultado para los veredictos seguro, sospechoso y malicioso

9.8. Usabilidad

Pendiente: Completar con la prueba de usabilidad de cinco participantes: perfil general de cada uno (edad y familiaridad con la tecnología, sin nombres), resultados de las tres métricas y frases textuales relevantes de los participantes al ver cada veredicto.

Tabla 9.9: Registro de la prueba de usabilidad (por completar)

P	**Edad**	T_{learn} **(s)**	**Comprendió**	**Finalizó**	**Observación**
1					
2					
3					
4					
5					

9.9. Cumplimiento de las métricas

El cuadro 9.10 reúne las diecisiete métricas del protocolo en el mismo orden del cuadro 7.4, para que puedan leerse en paralelo.

Tabla 9.10: Cumplimiento de las métricas del protocolo

Métrica	Criterio	Obtenido	Cumple
Clasificación URL o no URL	al menos 98 %	100 % en pruebas automatizadas; lectura con cámara pendiente (sección 9.6)	Parcial
Cobertura de pruebas	al menos 70 %	99,72 % de sentencias	Sí
Persistencia del historial	100 %	100 %	Sí
Reportes con carga completa	100 %	100 %	Sí
T_{scan} (percentil 90)	5 s o menos	Pendiente (sección 9.6)	
Tasa de detección (TDR)	al menos 95 %	Pendiente (sección 9.6)	
T_{F1} (percentil 90)	5 s o menos	214 ms y 488 ms con PhishTank, en red fija	Sí, en red fija
T_{total}	35 s o menos	31,3 s en el percentil 90 de las respuestas exitosas; 45 % de cortes	Parcial
Detección de maliciosos	al menos 90 %	Pendiente (sección 9.5)	
Falsos positivos	5 % o menos	Pendiente (sección 9.5)	
Geocodificación legible	al menos 85 %	Pendiente	
Story Card generada	al menos 90 %	Pendiente	
Comprensión del veredicto	al menos 90 %	Pendiente	
Finalización del flujo	al menos 85 %	Pendiente	
T_{learn}	60 s o menos	Pendiente	
T_{start}	3 s o menos	Pendiente	
Compatibilidad con Android 10 o superior	Sin errores críticos	Compila y se instala; verificación en uso pendiente	

Pendiente: Actualizar el cuadro 9.10 con cada medición pendiente. Donde una métrica no pueda medirse antes de la entrega, dejar la nota con la causa documentada en lugar de un guion.

9.10. Limitaciones de los resultados

Los resultados obtenidos hasta ahora tienen cinco limitaciones que deben considerarse al interpretarlos.

La primera es la disponibilidad del servidor. Durante el período de medición hubo días en que el servicio respondió con cortes generalizados: el 11 de octubre, tres llamadas consecutivas terminaron en HTTP 504 a los 35 s, incluida una URL que diez días antes respondía en 0,3 s desde caché. Además, la caché declarada en el contrato no funcionó en las pruebas, por lo que cada consulta pagó el costo completo de la cascada.

La segunda es el límite de 35 s de API Gateway, que corta casi la mitad de los análisis de URLs nuevas. Ese corte no depende de la aplicación, pero condiciona T_{total} y la detección.

La tercera es la muestra. El conjunto de Kaggle tiene etiquetas ruidosas y URLs antiguas, por lo que solo sirvió para medir tiempos y comportamiento operativo. Las cuotas de PhishTank y VirusTotal limitaron el tamaño de las corridas.

La cuarta es el entorno. Los tiempos de F1 se midieron en una computadora con red fija, y las pruebas en hardware se realizan sobre un único teléfono de gama de entrada.

La quinta es que el servidor solo concluye malicioso cuando F1 lo hace. La heurística y el comportamiento tienen como techo el veredicto sospechoso, y la aplicación compensa una parte de ese techo con la elevación por evidencia, de modo que la TDM depende en buena medida de qué amenazas conocen las listas de reputación en el momento de la prueba.

Pendiente: Agregar las limitaciones que surjan del banco de pruebas automatizado (por ejemplo, la diferencia entre decodificar imágenes fijas y leer con la cámara en vivo) y de la prueba de usabilidad.

CAPÍTULO 10

Conclusiones

1. Se desarrolló un prototipo funcional de aplicación móvil para Android que se coloca entre la lectura de un código QR y la apertura del enlace, y que analiza la URL sin visitarla nunca desde el teléfono. El prototipo integra de extremo a extremo el escaneo, la clasificación del contenido, el filtro F1 de reputación, la cascada de análisis del servidor y la presentación del veredicto, con lo que cubre el flujo completo planteado en el objetivo general.

2. El módulo de escaneo lee códigos QR con la cámara del dispositivo mediante `expo-camera` y clasifica su contenido con una precisión del 100 % en los 18 casos de prueba automatizada, separando los enlaces de las credenciales Wi-Fi, contactos, correos, teléfonos, coordenadas y texto. El cambio de librería respecto del protocolo redujo los conflictos entre dependencias nativas sin perder ninguna capacidad requerida.

   Pendiente: Completar esta conclusión con el tiempo de lectura y la tasa de detección del banco de pruebas automatizado, e indicar si se cumplió el criterio de 5 s en el percentil 90.

3. El flujo de interacción presenta el veredicto en cuatro categorías con dos niveles de detalle. La vista simple comunica la decisión en lenguaje llano y la vista extendida muestra los criterios reales que produjo el análisis. La regla de que ningún fallo produce un veredicto seguro, junto con la validación del puntaje, evita el error más grave del producto: un falso "seguro" con apariencia de correcto.

   Pendiente: Agregar los resultados de comprensión del veredicto, finalización del flujo y T_{learn} de la prueba de usabilidad.

4. Las funcionalidades secundarias quedaron implementadas: el historial conserva los últimos 20 análisis completos y los recupera sin conexión, el reporte construye una carga anónima y no pierde lo escrito ante un fallo, y la Story Card reúne la foto del entorno, el código cubierto con una capa opaca, el lugar y el veredicto. Las métricas automatizadas de persistencia y de carga del reporte se cumplieron al 100 %. El guardado del reporte depende de una corrección pendiente en el servidor.

   Pendiente: Completar con la geocodificación legible y la generación de la Story Card verificadas en el teléfono.

5. El filtro F1 se resolvió en el propio dispositivo con una lista local de dominios confiables y consultas en paralelo a Google Safe Browsing y PhishTank, y respondió con un percentil 90 de 488 ms sobre 400 URLs, muy por debajo del criterio de 5 s. Incorporar PhishTank multiplicó

por siete la detección de phishing del filtro sin producir falsos positivos en la muestra. Cuando F1 concluye, el servidor responde en alrededor de 5 s en lugar de ejecutar la cascada completa.

6. Las pruebas sobre amenazas activas mostraron que ningún filtro por sí solo es suficiente: Safe Browsing no detectó ninguna de 22 URLs de distribución de malware, la heurística detectó la que usaba una IP literal y solo el análisis de comportamiento detectó una carga maliciosa en un dominio de aspecto normal. Este resultado respalda empíricamente la decisión de organizar el análisis como una cascada de filtros complementarios.

7. El principal obstáculo para el desempeño del sistema completo no está en la aplicación, sino en el límite de 35 s de API Gateway, que cortó el 45 % de los análisis en la prueba de la cascada. La aplicación mitiga ese límite con un presupuesto de 90 s y un reintento, y degrada a incierto en lugar de inventar un veredicto, pero la solución de fondo corresponde al servidor.

CAPÍTULO 11

Recomendaciones

1. **Eliminar el corte de 35 s del servidor.** Es la mejora con mayor impacto sobre todo el sistema. Puede lograrse devolviendo primero el veredicto y entregando la captura de F2.3 por separado, usando un punto de acceso sin el límite de API Gateway o adoptando un esquema de consulta asíncrona en el que la aplicación pregunte periódicamente por el resultado.
2. **Reparar la caché de análisis.** El contrato declara una caché con vigencia de 7 días para los resultados sospechosos y maliciosos y de 24 horas para los demás, pero en las pruebas cada consulta repetida volvió a pagar el costo completo. Una caché funcional reduciría la latencia y el consumo de las cuotas de los servicios externos.
3. **Permitir que la heurística y el comportamiento concluyan malicioso en el servidor.** Hoy esos filtros tienen como techo el veredicto sospechoso. Si el servidor aplicara el mismo umbral de tres motores que usa la aplicación, la elevación por evidencia podría retirarse del cliente.
4. **Excluir de la lista local los servicios con contenido de usuarios.** Las pruebas mostraron que dominios confiables como `google.com` y `gitlab.com` alojan servicios donde cualquier persona puede subir archivos. Se recomienda que la lista local no acepte esos subdominios o rutas, para que sus URLs pasen por toda la cascada.
5. **Agregar URLhaus como fuente de F1.** Safe Browsing no cubrió la infraestructura de distribución de malware, que es justamente la especialidad de URLhaus. Integrarlo como cuarta fuente mejoraría la detección temprana de ese tipo de amenaza.
6. **Obtener una clave de aplicación de PhishTank** o incorporar OpenPhish como respaldo, para reducir los cortes por límite de consultas observados en las pruebas.
7. **Cifrar el historial local.** Aunque no contiene credenciales ni coordenadas, cifrarlo cumpliría por completo con la categoría de almacenamiento de OWASP MASVS y con lo planteado en el protocolo.
8. **Distinguir entre falta de conexión y falla del servidor.** Hoy ambos casos se comunican como incierto con el mismo mensaje. Detectar el estado de la red permitiría dar al usuario una indicación más precisa.
9. **Retomar la versión para iOS.** La configuración que no requiere macOS se conservó en el proyecto. Completarla requiere una computadora con macOS y una cuenta de desarrollador de Apple.

10. **Evolucionar el filtro heurístico hacia un modelo de aprendizaje automático** entrenado con conjuntos de URLs etiquetadas recientes, y evaluarlo con muestras frescas y no con conjuntos históricos.

11. **Realizar un estudio de usabilidad con una muestra representativa** y en condiciones reales de uso, para validar en el entorno móvil los hallazgos sobre advertencias de seguridad que la literatura obtuvo en navegadores de escritorio.

Pendiente: Revisar las recomendaciones cuando estén los resultados del banco de pruebas y de la prueba de usabilidad, por si surgen recomendaciones nuevas sobre la lectura o sobre la interfaz.

Bibliografía

[1] *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*, 3.ª ed., International Organization for Standardization, 2015. [En línea]. Disponible: https://www.iso.org/standard/62021.html

[2] *Information technology — Automatic identification and data capture techniques — QR code bar code symbology specification*, 4.ª ed., International Organization for Standardization, 2024. [En línea]. Disponible: https://www.iso.org/standard/83389.html

[3] K. Krombholz, P. Frühwirt, P. Kieseberg, I. Kapsalis, M. Huber, y E. Weippl, "QR code security: A survey of attacks and challenges for usable security," en *Human Aspects of Information Security, Privacy, and Trust (HAS 2014)*, ser. Lecture Notes in Computer Science, T. Tryfonas y I. Askoxylakis, Eds., vol. 8533. Cham, Switzerland: Springer, 2014, pp. 79–90.

[4] T. Vidas, E. Owusu, S. Wang, C. Zeng, L. F. Cranor, y N. Christin, "QRishing: The susceptibility of smartphone users to QR code phishing attacks," en *Financial Cryptography and Data Security (FC 2013 Workshops)*, ser. Lecture Notes in Computer Science, A. A. Adams, M. Brenner, y M. Smith, Eds., vol. 7862. Berlin, Germany: Springer, 2013, pp. 52–69.

[5] Insikt Group, "Security Challenges Rise as QR Code and AI-Generated Phishing Proliferate," Recorded Future, Inf. téc. CTA-2024-0718, jul. 2024. [En línea]. Disponible: https://www.recordedfuture.com/research/qr-code-and-ai-generated-phishing-proliferate

[6] Palo Alto Networks Unit 42, "Phishing on the edge of the web and mobile using QR codes," Unit 42 Threat Research, 2023. [En línea]. Disponible: https://unit42.paloaltonetworks.com/qr-codes-as-attack-vector/

[7] S. Sheng, B. Wardman, G. Warner, L. F. Cranor, J. Hong, y C. Zhang, "An empirical analysis of phishing blacklists," en *Proceedings of the 6th Conference on Email and Anti-Spam (CEAS 2009)*, Mountain View, CA, USA, 2009. [En línea]. Disponible: https://kilthub.cmu.edu/articles/journal_contribution/An_Empirical_Analysis_of_Phishing_Blacklists/6469805

[8] Anti-Phishing Working Group, "Phishing Activity Trends Report, 1st Quarter 2024," APWG, Inf. téc., 2024. [En línea]. Disponible: https://docs.apwg.org/reports/apwg_trends_report_q1_2024.pdf

[9] ——, "Phishing Activity Trends Report, 1st Quarter 2025," APWG, Inf. téc., 2025. [En línea]. Disponible: https://docs.apwg.org/reports/apwg_trends_report_q1_2025.pdf

[10] Google, "Google Safe Browsing — Overview (v5)," Google for Developers. [En línea]. Disponible: https://developers.google.com/safe-browsing

[11] PhishTank / Cisco Talos, "Developer Information," PhishTank. [En línea]. Disponible: https://phishtank.org/developer_info.php

[12] Verizon, "2024 Data Breach Investigations Report," Verizon Business, Inf. téc., 2024. [En línea]. Disponible: https://www.verizon.com/business/resources/reports/2024-dbir-data-breach-investigations-report.pdf

[13] F. Salahdine y N. Kaabouch, "Social Engineering Attacks: A Survey," *Future Internet*, vol. 11, n.º 4, p. 89, abr. 2019. [En línea]. Disponible: https://www.mdpi.com/1999-5903/11/4/89

[14] D. Sahoo, C. Liu, y S. C. H. Hoi, "Malicious URL detection using machine learning: A survey," *arXiv preprint arXiv:1701.07179*, 2017. [En línea]. Disponible: https://arxiv.org/abs/1701.07179

[15] T. Berners-Lee, R. T. Fielding, y L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," RFC 3986, Internet Engineering Task Force, 2005. [En línea]. Disponible: https://www.rfc-editor.org/rfc/rfc3986

[16] MISP Project, "MISP — Open Source Threat Intelligence Platform & Open Standards For Threat Information Sharing," MISP Project. [En línea]. Disponible: https://www.misp-project.org/

[17] abuse.ch, "URLhaus — API," URLhaus Documentation. [En línea]. Disponible: https://urlhaus.abuse.ch/api/

[18] Google LLC, "VirusTotal API v3 — Overview," VirusTotal Documentation. [En línea]. Disponible: https://docs.virustotal.com/reference/overview

[19] J. Ma, L. K. Saul, S. Savage, y G. M. Voelker, "Beyond blacklists: Learning to detect malicious web sites from suspicious URLs," en *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09)*, Paris, France, 2009, pp. 1245–1254.

[20] A. Le, A. Markopoulou, y M. Faloutsos, "PhishDef: URL names say it all," en *Proceedings of IEEE INFOCOM 2011*, Shanghai, China, 2011, pp. 191–195. [En línea]. Disponible: https://arxiv.org/abs/1009.2275

[21] M. Siddhartha, "Malicious URLs dataset," Kaggle, 2021. [En línea]. Disponible: https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset

[22] M. Sokolova y G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, n.º 4, pp. 427–437, jul. 2009.

[23] Apple Inc., "VNDetectBarcodesRequest — Vision framework," Apple Developer Documentation. [En línea]. Disponible: https://developer.apple.com/documentation/vision/vndetectbarcodesrequest

[24] ——, "Technical Note TN2325: AV Foundation iOS machine readable code detection FAQ," Apple Developer Documentation. [En línea]. Disponible: https://developer.apple.com/library/archive/technotes/tn2325/_index.html

[25] Google, "Scan barcodes with ML Kit on Android," Google for Developers. [En línea]. Disponible: https://developers.google.com/ml-kit/vision/barcode-scanning/android

[26] M. Rousavy, "VisionCamera — The Barcode Scanner," react-native-vision-camera Documentation. [En línea]. Disponible: https://react-native-vision-camera.com/docs/guides/code-scanning

[27] Expo, "Camera (expo-camera), Expo SDK 57," Expo Documentation. [En línea]. Disponible: https://docs.expo.dev/versions/v57.0.0/sdk/camera/

[28] Meta Platforms, "About the New Architecture," React Native Documentation. [En línea]. Disponible: https://reactnative.dev/architecture/landing-page

[29] Expo, "Introduction to Expo Router," Expo Documentation. [En línea]. Disponible: https://docs.expo.dev/router/introduction/

[30] ——, "EAS build," Expo Documentation. [En línea]. Disponible: https://docs.expo.dev/build/introduction/

[31] OWASP Foundation, "OWASP Mobile Application Security Verification Standard (MASVS) v2.1," OWASP Mobile Application Security, 2024. [En línea]. Disponible: https://mas.owasp.org/MASVS/

[32] Apple Inc., "NSAppTransportSecurity — Information Property List," Apple Developer Documentation. [En línea]. Disponible: https://developer.apple.com/documentation/bundleresources/information-property-list/nsapptransportsecurity

[33] ——, "Apple Platform Security," Apple Inc., Inf. téc., 2024. [En línea]. Disponible: https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf

[34] Google, "Network security configuration," Android Developers. [En línea]. Disponible: https://developer.android.com/privacy-and-security/security-config

[35] M. Rousavy, "react-native-mmkv — The fastest key/value storage for React Native," GitHub Repository. [En línea]. Disponible: https://github.com/mrousavy/react-native-mmkv

[36] ——, "StorageBenchmark — Benchmark App to compare different storage libraries (MMKV, AsyncStorage, WatermelonDB, RealmDB, SQLite)," GitHub Repository. [En línea]. Disponible: https://github.com/mrousavy/StorageBenchmark

[37] Expo, "SQLite (expo-sqlite), Expo SDK 57," Expo Documentation. [En línea]. Disponible: https://docs.expo.dev/versions/v57.0.0/sdk/sqlite/

[38] OpenPhish, "OpenPhish — Phishing Intelligence," OpenPhish. [En línea]. Disponible: https://openphish.com/

[39] Apple Inc., "Requesting authorization to use location services," Apple Developer Documentation. [En línea]. Disponible: https://developer.apple.com/documentation/corelocation/requesting-authorization-to-use-location-services

[40] Google, "Request location permissions," Android Developers. [En línea]. Disponible: https://developer.android.com/develop/sensors-and-location/location/permissions

[41] Apple Inc., "CLGeocoder — Core Location," Apple Developer Documentation. [En línea]. Disponible: https://developer.apple.com/documentation/corelocation/clgeocoder

[42] Mapbox, "Geocoding API — Mapbox Search Service," Mapbox API Documentation. [En línea]. Disponible: https://docs.mapbox.com/api/search/geocoding/

[43] L. F. Cranor, "A framework for reasoning about the human in the loop," en *Proceedings of the 1st Conference on Usability, Psychology, and Security (UPSEC '08)*, San Francisco, CA, USA, 2008, p. 1. [En línea]. Disponible: https://www.usenix.org/legacy/event/upsec08/tech/full_papers/cranor/cranor.pdf

[44] S. Egelman, L. F. Cranor, y J. Hong, "You've been warned: An empirical study of the effectiveness of web browser phishing warnings," en *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*, Florence, Italy, 2008, pp. 1065–1074.

[45] D. Akhawe y A. P. Felt, "Alice in Warningland: A large-scale field study of browser security warning effectiveness," en *Proceedings of the 22nd USENIX Security Symposium*, Washington, D.C., USA, 2013, pp. 257–272. [En línea]. Disponible: https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/akhawe

[46] Amazon Web Services, "AWS Lambda — Developer Guide," Amazon Web Services Documentation. [En línea]. Disponible: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

[47] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Sequim, WA, USA: Blue Hole Press, 2010.

[48] J. Nielsen y T. K. Landauer, "A mathematical model of the finding of usability problems," en *Proc. INTERACT '93 and CHI '93 Conf. Human Factors in Computing Systems*, Amsterdam, Netherlands, 1993, pp. 206–213.

[49] R. J. Hyndman y Y. Fan, "Sample quantiles in statistical packages," *The American Statistician*, vol. 50, n.º 4, pp. 361–365, 1996.

[50] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, y W. Joosen, "Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation," en *Proc. Network and Distributed System Security Symposium (NDSS 2019)*, San Diego, CA, USA, 2019.

ANEXO A

Evidencia de las mediciones

Cada número del capítulo de resultados proviene de una corrida cuya salida se conserva en la carpeta `RESULTADOS` del repositorio del proyecto. El cuadro A.1 indica dónde se encuentra la evidencia de cada medición.

Tabla A.1: Ubicación de la evidencia en el repositorio

Medición	**Archivos**
Cobertura y suite completa	`fase0/0.1_cobertura.txt`, `fase0/0.5_suite_completa.txt`
Clasificación de contenido	`fase0/0.2_clasificacion.txt`, `fase0/0.2_clasificacion_casos.txt`
Persistencia del historial	`fase0/0.3_persistencia.txt`, `fase0/0.3_persistencia_casos.txt`
Carga de los reportes	`fase0/0.4_reportes.txt`, `fase0/0.4_reportes_casos.txt`
Entorno y tipos	`fase0/0.5_entorno.txt`, `fase0/0.5_typecheck.txt`
Compilación en EAS	`fase1/1.2_build_apk.txt`, `fase1/1.2_historial_builds.txt`
Ficha del teléfono	`fase1/1.4_ficha_telefono.txt`
Integración con la cascada	`fase3/3.0_resumen_verificacion.txt`, `fase3/3.0_verificacion_contrato.json`
F1 en vivo	`fase4_5/f1_verificacion_en_vivo.txt`, `fase4_5/f1_resumen.txt`
Elevación por evidencia	`fase4_5/elevacion_en_vivo.txt`
Muestras de Kaggle	`muestras/kaggle/resumen.md` y archivos de resultados
Cobertura de filtros con URLhaus	`muestras/hallazgo_cobertura_f1.md`

Pendiente: Agregar las rutas de la evidencia del banco de pruebas automatizado, de la verificación en dispositivo, de la muestra verificada para TDM y TFP, y de la prueba de usabilidad.

ANEXO B

Resumen del contrato de interfaz

B.1. Análisis de una URL

La aplicación solicita el análisis con una petición `POST /analyze` que envía la URL extraída del código y la conclusión del filtro F1:

```
{
  "url": "https://ejemplo.com",
  "resultadoF1": "INCIERTO"
}
```

El campo `resultadoF1` admite los valores `SEGURO`, `MALICIOSO` e `INCIERTO`. Con los dos primeros el servidor corta la cascada y devuelve ese veredicto. Con el tercero consulta la caché y, si no hay un resultado vigente, ejecuta F2.1 con trece criterios heurísticos, corta con `SOSPECHOSO` si el puntaje supera 60 y, en caso contrario, ejecuta F2.2 consultando URLScan.io y VirusTotal en paralelo. Si F2.2 obtiene una captura del sitio, F2.3 la guarda y devuelve una dirección firmada válida por una hora. Un ejemplo real de respuesta, verificado el 20 de septiembre de 2026, es el siguiente:

```
{
  "url": "http://192.168.1.1/login.php",
  "dominio": "192.168.1.1",
  "veredicto_final": "PENDIENTE",
  "score_heuristico": 60,
  "criterios_f2_1": [
    { "criterio": "ip_literal", "peso": 30,
      "detalle": "Host es una IP: 192.168.1.1" },
    { "criterio": "sin_https", "peso": 15,
      "detalle": "La URL no usa HTTPS" },
    { "criterio": "palabra_clave_sospechosa", "peso": 15,
      "detalle": "Palabra detectada: \"login\"" }
```

```
  ],
  "indicadores_f2_2": []
}
```

El campo `veredicto_final` puede ser `SEGURO`, `MALICIOSO`, `SOSPECHOSO` o `PENDIENTE`, y el campo `filtro_decisivo` indica si decidió F1, F2.1, F2.2 o ninguno.

B.2. Reporte de un código

El reporte se envía con una petición `POST /report` que incluye la URL, un motivo opcional y, si existe, el identificador del análisis. El servidor guarda el reporte en su propia base de datos. Como PhishTank no ofrece una API pública para enviar URLs, el reporte no se reenvía automáticamente a listas colaborativas externas.

```
{
  "url": "https://ejemplo-malicioso.xyz",
  "motivo": "Formulario pidiendo credenciales bancarias"
}
```

ANEXO C

Códigos QR de prueba

El cuadro C.1 muestra los contenidos usados para verificar la clasificación y el cuadro C.2 los usados para demostrar los veredictos. Los puntajes del segundo cuadro fueron verificados contra el servidor el 20 de septiembre de 2026 y pueden cambiar si se ajustan los pesos de la heurística.

Tabla C.1: Contenidos de ejemplo y pantalla esperada

Contenido del QR	**Resultado esperado**
`https://menu.cafeaurora.com/es`	Análisis
`cafeaurora.com/menu`	Análisis, se antepone `https`
`WIFI:T:WPA2;S:MiRed;P:clave123;;`	Contenido no navegable, red Wi-Fi
`mailto:hola@ejemplo.com`	Contenido no navegable, correo
`tel:+50255512346`	Contenido no navegable, teléfono
`geo:14.6349,-90.5069`	Contenido no navegable, ubicación
Contacto en formato vCard	Contenido no navegable, contacto
`Mesa 12`	Contenido no navegable, texto

Tabla C.2: Códigos de demostración de los veredictos

URL	**Puntaje**	**Veredicto**	**Señales**
`https://github.com`	0	Seguro	Ninguna
`http://banco-lndustrial-seguro.top/acceso`	40	Sospechoso	Dominio `.top` y sin HTTPS
`http://192.168.1.1/login.php`	60	Malicioso	IP literal, sin HTTPS y la palabra *login*

ANEXO D

Protocolo de verificación en dispositivo

El protocolo se ejecuta en una sola sesión de aproximadamente cincuenta minutos, con la aplicación recién instalada después de desinstalar cualquier versión anterior.

1. **Arranque en frío y onboarding.** Verificar que el onboarding aparece la primera vez, que solicita los permisos de cámara y ubicación con su explicación y que no vuelve a aparecer al reiniciar.
2. **Análisis real.** Escanear los tres códigos de demostración y verificar el veredicto, el medidor y los criterios del detalle. Escanear una URL nueva y verificar el aviso de espera prolongada.
3. **Geocodificación inversa.** Al aire libre y con el permiso concedido, escanear en diez ubicaciones distintas y anotar si el nombre del lugar y la ciudad son legibles. Reiniciar la aplicación y verificar que la preferencia de ubicación se conserva.
4. **Story Card.** Generar diez tarjetas con distintos veredictos, verificar los cuatro elementos y compartir o guardar cada una. Intentar escanear con un segundo teléfono el código cubierto de la tarjeta; si se lee, registrar un fallo grave.
5. **Camino sin conexión.** Activar el modo avión, escanear un código y verificar que el resultado es incierto con el aviso de degradación, sin que la aplicación se detenga. Verificar que el historial sigue disponible.
6. **Teclado y áreas seguras.** Abrir el formulario de reporte y verificar que el teclado no tapa los campos y que el contenido respeta los bordes de la pantalla.

En cada paso se registra qué se observó, además de si se aprobó o no.

Glosario

análisis heurístico Evaluación del riesgo de una URL a partir de su estructura, sin consultar bases externas ni visitar el sitio. 1

API Interfaz de programación de aplicaciones. Conjunto de operaciones que un servicio ofrece a otros programas. 1

arranque en frío Retraso de la primera solicitud a una función sin servidor que no se ha ejecutado recientemente, mientras se inicializa su entorno. 1

cascada de filtros Organización del análisis en filtros sucesivos, en la que cada uno se ejecuta solo si el anterior no llegó a una conclusión. 1

endpoint Dirección de un servicio web a la que se envían peticiones, como el servicio de análisis o el de reportes. 1

entorno aislado Entorno controlado donde un sitio se carga y analiza sin exponer el dispositivo del usuario. 1

falso negativo Caso malicioso que no se clasifica como tal. 1

falso positivo Caso legítimo clasificado como malicioso. 1

geocodificación inversa Conversión de coordenadas geográficas en una descripción legible, como el nombre de un lugar. 1

IOC Indicador de compromiso. Dato observable, como un dominio, una URL o una dirección IP, asociado a una actividad maliciosa. 1

lista negra Base de datos de URLs, dominios o direcciones IP identificadas como maliciosas. 1

onboarding Pantallas iniciales que presentan la aplicación y solicitan los permisos necesarios la primera vez que se usa. 1

percentil 90 Valor por debajo del cual se encuentra el 90 % de las observaciones de una muestra. 1

phishing Técnica de ingeniería social en la que un atacante suplanta a una entidad de confianza para obtener información sensible o dirigir a la víctima hacia contenido malicioso. 1

quishing Variante del phishing que distribuye un enlace malicioso dentro de un código QR. 1

sin servidor Modelo de despliegue en la nube en el que el proveedor asigna recursos por solicitud y no se administra un servidor fijo. 1

Story Card Imagen compartible que genera QRisk para advertir a otras personas sobre un código QR. 1

threat intelligence Disciplina que recolecta, analiza y comparte información sobre amenazas de seguridad. 1

TLD Dominio de nivel superior. Última parte de un nombre de dominio, como .com o .top. 1

URL Localizador uniforme de recursos. Cadena que identifica un recurso en la web, con la sintaxis definida en el RFC 3986. 1

Lista de Símbolos

T_{F1}	Tiempo que tarda el filtro F1 en obtener su conclusión
T_{learn}	Tiempo que tarda un usuario sin instrucciones en completar su primer escaneo
T_{scan}	Tiempo desde que el lector está listo hasta que entrega el contenido del código
T_{start}	Tiempo desde que se toca el ícono hasta que el escáner está listo
T_{total}	Tiempo desde el envío de la URL hasta el veredicto final
p90	Percentil 90
TDM	Tasa de detección de URLs maliciosas
TDR	Tasa de detección de códigos QR
TFP	Tasa de falsos positivos